Linux : initiation et préparation LPI 101

Tous les cours — Sommaire

01. Comprendre GNU/Linux et le logiciel libre — p. 3

02. Choisir et reconnaître une distribution Linux — p. 6

03. Comprendre les composants du système — p. 8

04. Préparer une installation Linux — p. 11

05. Installer Debian dans une machine virtuelle — p. 14

06. Accéder à la VM et préparer sa session — p. 18

07. Comprendre et utiliser une commande — p. 21

08. Se déplacer et comprendre les chemins — p. 23

09. Lire et identifier des fichiers — p. 27

10. Créer, copier, déplacer et supprimer — p. 29

11. Utiliser les jokers et rechercher — p. 32

12. Comprendre les liens physiques et symboliques — p. 35

13. Archiver et compresser — p. 37

14. Variables, environnement et historique — p. 40

15. Flux, redirections et tubes — p. 42

16. Filtrer et transformer du texte — p. 46

17. Rechercher avec les expressions régulières — p. 50

18. Modifier un fichier avec nano et vi — p. 55

19. Utilisateurs, groupes et droits administrateur — p. 60

20. Lire et modifier les permissions — p. 66

21. Droits par défaut et bits spéciaux — p. 71

22. Observer et contrôler les processus — p. 74

23. Comprendre les priorités des processus — p. 79

24. Se repérer dans l’arborescence Linux — p. 81

25. Concevoir un partitionnement simple — p. 85

26. Créer des partitions et des systèmes de fichiers — p. 87

27. Monter un système de fichiers et utiliser fstab — p. 92

28. Mesurer l’espace, la mémoire et les inodes — p. 96

29. Vérifier et entretenir les systèmes de fichiers — p. 99

30. Installer et gérer les paquets Debian — p. 102

31. Reconnaître RPM, DNF, YUM et Zypper — p. 106

32. Comprendre les bibliothèques partagées — p. 109

33. Identifier le matériel et les modules du noyau — p. 112

34. Observer les événements matériels — p. 117

35. Démarrage, arrêt et cibles systemd — p. 120

36. Gérer les services avec systemd — p. 126

37. Consulter les journaux et diagnostiquer — p. 130

38. Comprendre GRUB et la récupération — p. 135

39. Se connecter à distance avec SSH — p. 138

C01. Donner des droits précis avec les ACL — p. 143

C02. Comprendre et manipuler LVM — p. 146

C03. Écrire un premier script Bash — p. 150

C04. Découvrir les conditions et les boucles — p. 152

C05. Lire des scripts utilitaires — p. 154

C06. Découvrir Linux sous Windows avec WSL — p. 155

C07. Découvrir Cloud-init — p. 160

C08. Personnaliser son terminal — p. 165

C09. Repères pour poursuivre la préparation LPI 101 — p. 168

01. Comprendre GNU/Linux et le logiciel libre

Objectif Distinguer noyau, système et logiciel libre ; comprendre les origines de Linux.

Hedgedoc complet sur l'histoire de Linux par Bastien TRAVERSE - https://hedgedoc.dawan.fr/01dOettVSr63m4VojmMegA

Vidéo du cours : ouvrir la vidéo en ligne

Introduction

Avant d'utiliser Linux, il est essentiel d'en comprendre l'origine, la philosophie et ce qui le distingue des autres systèmes d'exploitation. Linux n'est pas seulement un logiciel : c'est un **écosystème** basé sur la **liberté**, la **collaboration** et la **transparence**.

Historique de GNU/Linux

Dans les années 1980, **Richard Stallman** lance le projet **GNU** (GNU's Not Unix) avec un objectif clair : créer un système d'exploitation libre, compatible avec Unix, où **tout le monde peut lire, modifier et redistribuer le code**.

Cependant, le projet GNU n'avait pas de noyau fonctionnel. En 1991, **Linus Torvalds**, un étudiant finlandais, publie les premières versions du noyau **Linux**. Linux adopte la licence GNU GPL en 1992.

L'association du projet GNU et du noyau Linux donnera naissance à ce qu'on appelle souvent **GNU/Linux**.

Pourquoi Linux ?

Linux s'est imposé grâce à plusieurs avantages :

- **Stabilité et fiabilité** : utilisé sur la majorité des serveurs web, routeurs, systèmes embarqués…

- **Sécurité** : structure des permissions stricte et mises à jour rapides.
- **Performance** : léger, optimisable et capable de tourner sur de vieilles machines.
- **Flexibilité** : adapté à tous les usages — serveur, poste de travail, embarqué, cloud, etc.
- **Gratuit et libre** : la plupart des distributions sont entièrement gratuites.

Aujourd'hui, Linux est partout : dans les serveurs, les téléphones Android, les box internet, les voitures et même les supercalculateurs.

Comparaison avec Windows et macOS

Caractéristique	Linux	Windows	macOS
Licence	Libre et gratuit	Propriétaire	Propriétaire
Code source	Ouvert	Fermé	Fermé
Sécurité	Dépend des mises à jour, de la configuration et des usages	Même principe	Même principe
Performances	Dépendent du matériel et de la charge	Même principe	Même principe
Utilisation	Serveurs, dev, réseaux	Bureautique, jeux	Création, design
Personnalisation	Très élevée	Limitée	Faible
Écosystème	Communautés et entreprises	Éditeur, partenaires et communautés	Éditeur, partenaires et communautés

---

Philosophie du Libre

Le **logiciel libre** ne signifie pas "gratuit" mais "libre d'utilisation". La **Free Software Foundation (FSF)**, fondée par Richard Stallman, définit **4 libertés fondamentales** :

1. La liberté d'**exécuter** le programme pour tout usage.
2. La liberté d'**étudier et modifier** son fonctionnement, avec accès au code source.
3. La liberté de **redistribuer des copies**.
4. La liberté de **distribuer des versions modifiées**, avec accès au code source.

Ces principes garantissent une **autonomie technologique** et favorisent l'innovation collective.

L’Open Source Initiative (OSI)

Dans les années 1990, certains acteurs ont voulu rendre le concept plus “entrepreneurial”. L’**Open Source Initiative (OSI)** reprend les mêmes principes, mais avec une approche **plus pragmatique** et orientée vers les entreprises.

Ainsi, “**logiciel libre**” (Free Software) et “**Open Source**” désignent souvent la même chose, mais avec **deux philosophies différentes** :

- **Libre** → liberté de l’utilisateur (philosophie éthique)
- **Open Source** → efficacité et transparence (philosophie pragmatique)

Questions interactives

1. **Vrai ou faux :** Linux a été créé par Richard Stallman.

 Indice : Stallman a fondé le projet GNU, mais pas le noyau.

1. Quelle est la différence principale entre “logiciel libre” et “open source” ?

 Une réponse en une phrase suffit.

1. Donne un **exemple d’utilisation** de Linux dans la vie quotidienne (hors serveur).
2. Pourquoi penses-tu que les entreprises préfèrent souvent utiliser Linux sur leurs serveurs plutôt que Windows ?

Vidéo du cours : ouvrir la vidéo en ligne

À retenir

- Linux est né de la rencontre entre le projet **GNU** et le noyau créé par **Linus Torvalds**.
- C’est un système **libre, ouvert et collaboratif**.
- La **Free Software Foundation (FSF)** et l’**Open Source Initiative (OSI)** défendent deux visions complémentaires du logiciel libre.
- Linux domine aujourd’hui les **serveurs**, le **cloud** et les **systèmes embarqués**.

02. Choisir et reconnaître une distribution Linux

Objectif Repérer les grandes familles de distributions et leurs usages.

Introduction

Le **noyau Linux** est le cœur du système, mais il n'est pas suffisant seul. Une **distribution Linux** rassemble le noyau, les outils GNU, une interface (graphiques ou non), un système de paquets et divers logiciels.

Chaque distribution est donc une **version complète** du système Linux, adaptée à un public ou un usage particulier.

- **Liste distribution Linux**
https://upload.wikimedia.org/wikipedia/commons/9/96/Liste_des_distributions_Linux.svg

- **Liste distribution Unix**
https://upload.wikimedia.org/wikipedia/commons/c/c3/Unix_history-simple.fr.svg

---

Les grandes familles

- **Debian / Ubuntu / Linux Mint** → Simples, stables, très répandues.
- **Red Hat / Fedora / Rocky / AlmaLinux** → Orientation professionnelle, serveurs d'entreprise.
- **SUSE / openSUSE** → Outils d'administration puissants (YaST), souvent utilisés en entreprise.
- **Autres** : Arch (pour experts), Kali (sécurité), Raspberry Pi OS (éducation, embarqué).

Serveur vs Desktop

Type	Objectif	Interface	Exemple
Serveur	Fournir des services (web, mail, base de données)	En général sans interface graphique	Debian Server, Rocky Linux
Desktop	Utilisation personnelle, bureautique	Interface complète (GNOME, KDE…)	Ubuntu Desktop, Linux Mint

---

 Pour ce cours, nous utiliserons **Debian / Ubuntu**, idéales pour apprendre les bases.

Questions interactives

1. À quoi sert une distribution Linux ?
2. Cite deux distributions basées sur Debian.
3. Quelle différence entre une version “serveur” et une version “desktop” ?
4. Pourquoi Debian/Ubuntu sont-elles souvent utilisées en formation ?

À retenir

- Une **distribution** = noyau Linux + outils + gestionnaire de paquets + interface.

• Les principales familles sont **Debian**, **Red Hat**, et **SUSE**.

• Les distributions peuvent viser un usage **serveur** ou **desktop**.

• **Debian/Ubuntu** sont recommandées pour débuter et pour la formation.

03. Comprendre les composants du système

Objectif Situer le noyau, les pilotes, les programmes et les services.

Repères LPI 101 : 101.1, 101.2.

Introduction

Un système Linux repose sur plusieurs **couches logicielles** qui coopèrent pour faire fonctionner l'ensemble de la machine. Comprendre cette architecture permet de mieux saisir le rôle de chaque composant et de savoir où agir lorsqu'un problème survient.

Les principales couches du système

1. **Le noyau (Kernel)**

• C'est le cœur du système.

• Il gère le matériel (processeur, mémoire, disques, périphériques) et assure la communication entre le matériel et les programmes.

1. **Les pilotes (Drivers)**

• Ce sont les modules qui permettent au noyau de reconnaître et contrôler les composants matériels.

• Ils sont souvent intégrés au noyau ou chargés dynamiquement au démarrage.

1. **Les processus et services**

• Un **processus** est un programme en cours d'exécution.

• Certains tournent en arrière-plan (appelés **services** ou **démons**).

• Ils assurent des tâches continues : réseau, impression, gestion du temps, etc.

1. **Les utilisateurs et groupes**

• Chaque action sous Linux est réalisée par un utilisateur.

• Le compte **root** a tous les droits, les autres ont des permissions limitées.

• Les **groupes** permettent de gérer les droits collectivement.

1. **L'espace utilisateur (User Space)**

• C'est la partie visible pour l'utilisateur : commandes, applications, interfaces graphiques.

• Elle utilise des bibliothèques et communique avec le noyau par des appels système.

---

systemd et l’initialisation du système

Lors du démarrage, Linux charge le noyau puis lance un programme d’initialisation.
Aujourd’hui, c’est souvent **systemd** qui s’occupe :

• de démarrer les services,

• de gérer les sessions,

• et de contrôler l’arrêt ou le redémarrage du système.

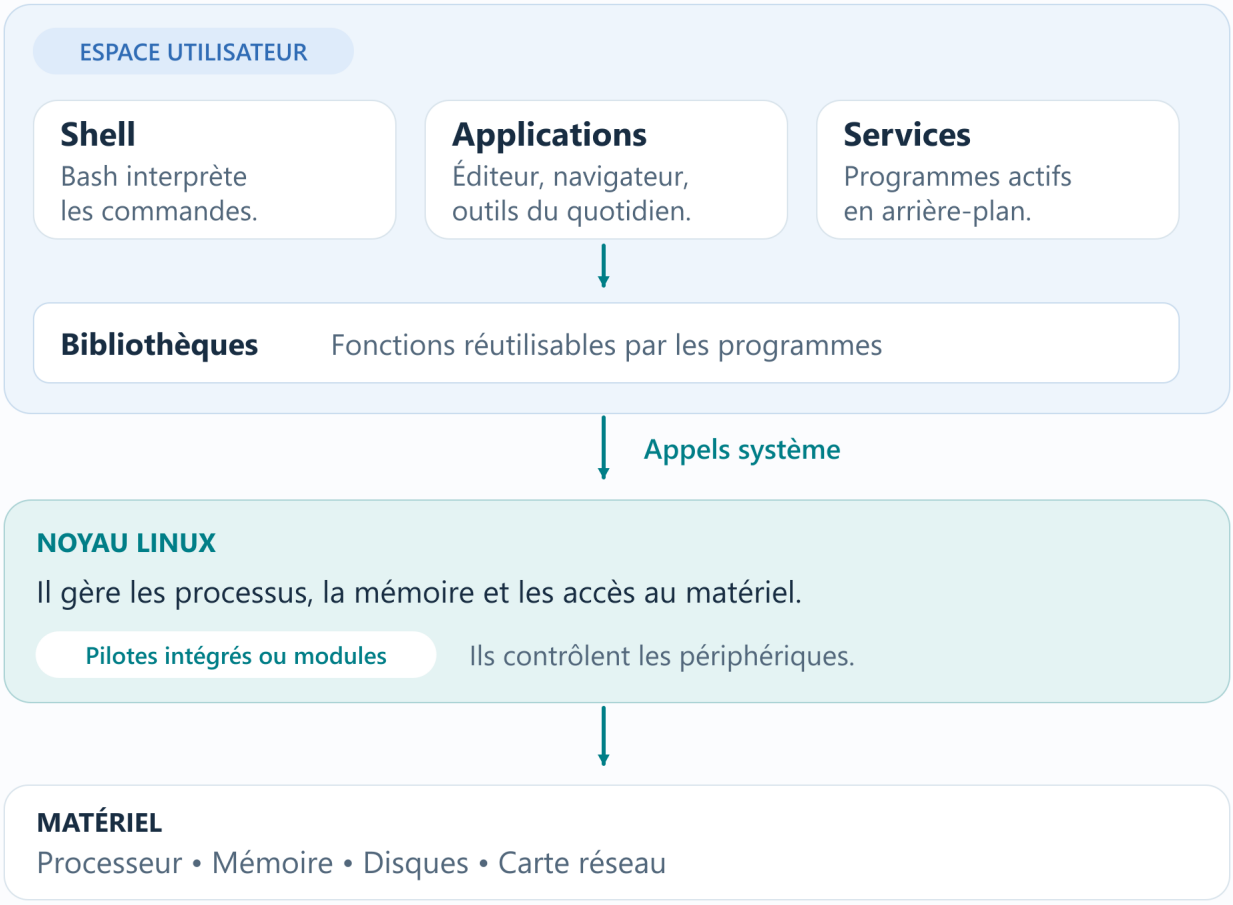
Anciennement, ce rôle était assuré par **SysVinit**.

Lire le système comme un ensemble

LPI 101 / REPÈRE VISUEL

Du programme au matériel

Les couches de Linux, du plus visible au plus proche de la machine.



Lecture simplifiée : les programmes demandent au noyau d'agir sur la machine.

Les couches de Linux : programmes et bibliothèques dans l'espace utilisateur, noyau et pilotes, puis matériel.

Le terminal affiche une session ; le **shell** interprète les commandes. Bash est un shell courant. Un pilote peut être intégré au noyau ou fourni comme module chargeable.

Le noyau est le passage entre les programmes et le matériel ; les services font partie de l'espace utilisateur.

À retenir

* Le **noyau** est le cœur du système et communique avec le matériel. * Les **pilotes** permettent la reconnaissance et le contrôle des périphériques. * Les **processus** exécutent les programmes et certains tournent en arrière-plan (services/démons). * Les **utilisateurs et groupes** définissent les droits d'accès. * **systemd** gère le démarrage, les services et l'arrêt du système moderne.

04. Préparer une installation Linux

Objectif Choisir une image d'installation et reconnaître les étapes du démarrage.

Repères LPI 101 : 101.2, 102.1, 102.2.

Télécharger une ISO officielle

Une **ISO** est une image disque contenant tout le nécessaire pour installer un système d'exploitation. Elle peut être gravée sur un DVD ou copiée sur une clé USB pour démarrer l'installation.

Les principaux types d'ISO

Type d'image	Description	Utilisation
NetInstall	Image généralement plus petite qu'un DVD complet ; taille variable selon la version. Télécharge les paquets depuis Internet pendant l'installation.	Idéale pour une installation personnalisée ou sur machine virtuelle.
CD/DVD complet	Contient une sélection importante de paquets ; d'autres peuvent demander un téléchargement.	À utiliser sans connexion Internet.
Live	Permet de tester Linux sans l'installer (exécution depuis la RAM).	Utile pour découvrir l'environnement avant installation.
Minimal / Cloud / ARM	Versions destinées à des environnements spécifiques (serveurs, cloud, architectures ARM).	Pour les usages avancés.

Liens officiels :

- Debian : https://www.debian.org/distrib/netinst
- Ubuntu : https://ubuntu.com/download

BIOS vs UEFI

Le **BIOS** (Basic Input/Output System) et **UEFI** (Unified Extensible Firmware Interface) sont des programmes de bas niveau qui démarrent le matériel et chargent le système d'exploitation.

Caractéristique	BIOS	UEFI
Période	Ancien (années 1980)	Moderne (depuis 2010)
Interface	Souvent textuelle	Textuelle ou graphique selon le firmware
Partitionnement courant	MBR ou GPT selon le système	GPT habituellement ; partition système EFI
Chargeur d'amorçage	Stocké dans le MBR	Stocké dans la partition EFI
Sécurité	Aucune	Secure Boot (signature numérique)

Les PC récents utilisent UEFI, mais le BIOS reste courant sur les machines virtuelles ou les anciens ordinateurs.

Le chargeur d'amorçage (GRUB)

Le **chargeur d'amorçage** (ou **bootloader**) est le programme lancé juste après le BIOS/UEFI. Son rôle est de **charger le noyau Linux** et de **démarrer le système**.

Le plus utilisé sous Linux est **GRUB 2**.

Étapes de démarrage :

1. Le BIOS ou l'UEFI charge GRUB depuis le disque.
2. GRUB affiche un menu si plusieurs OS sont présents.
3. GRUB charge le noyau Linux (`vmlinuz`) et l'image initrd.
4. Le noyau initialise le matériel et lance le processus d'initialisation, qui démarre notamment les services.

Commandes utiles :

```
sudo grub-install /dev/sda    # Exemple BIOS sur disque de laboratoire ; ne pas lancer sans
     identifier la cible
sudo update-grub              # Met à jour la configuration
sudo nano /etc/default/grub   # Modifie les options du menu
```

Pour restaurer GRUB, on peut démarrer sur un **Live CD** et réinstaller le chargeur manuellement.

Pour aller plus loin :

102.2 Leçon 1

À retenir

- Une **ISO** contient l'image du système à installer (NetInstall, Live, complète, etc.). - Le **partitionnement** organise l'espace disque (/, /home, swap…). - **UEFI** remplace le **BIOS** sur les systèmes récents. - Le **chargeur d'amorçage GRUB** lance le noyau Linux au démarrage. - En cas de problème, GRUB peut être réparé depuis un **Live CD**.

Démarrage UEFI L'installation de GRUB en UEFI utilise la partition système EFI montée et des options adaptées. La commande BIOS sur `/dev/sda` ne constitue pas une procédure universelle de réparation.

05. Installer Debian dans une machine virtuelle

Objectif Créer une VM de formation, installer Debian et comprendre son environnement.

Repères LPI 101 : 102.6.

Créer une machine virtuelle sous VirtualBox

VirtualBox permet d'exécuter une machine virtuelle (VM) Linux dans un environnement isolé sur ton ordinateur. Cela évite de modifier le système principal.

Étapes de création :

1. **Nom et type :** choisis un nom (ex. *Debian-LPI*) et sélectionne

• Type : **Linux**

• Version : **Debian (64-bit)**

2. **Mémoire vive (RAM) :** Alloue **2 à 4 Go** selon ta configuration.
3. **Processeur (CPU) :** Minimum **2 cœurs**, idéalement **4** pour de meilleures performances.
4. **Disque dur virtuel :**

• Type : VDI (VirtualBox Disk Image)

• Taille dynamique

• Taille : **20 à 40 Go**

5. **Réseau :**

• **NAT** : pour accéder à Internet depuis la VM

• **Host-Only** : pour communiquer avec la machine hôte sans accès Internet

• Les deux peuvent être combinés pour des scénarios plus avancés (NAT + Host-Only).

Étapes de l'installation Debian

Une fois la VM créée, démarre-la en sélectionnant ton **image ISO Debian** (NetInstall, DVD, etc.).

Installation en images : Debian 13 trixie : Install : Server World

Déroulement de l’installation (mode graphique ou texte) :

1. **Langue et disposition du clavier**
2. **Nom de la machine** (hostname)
3. **Configuration du réseau**

• DHCP automatique (recommandé en NAT)

• Ou configuration manuelle (IP fixe, DNS, passerelle)

4. **Création des utilisateurs :**

• **root** (administrateur)

• **utilisateur standard**

5. **Partitionnement du disque :**

• Automatique recommandé pour débuter

• Possibilité de séparer `/home`, `/var`, si besoin

6. **Sélection des paquets :**

• **Environnement graphique** (GNOME, XFCE, etc.)

• **Utilitaires standard du système**

• Serveur SSH (optionnel mais utile pour l’administration à distance)

7. **Installation du chargeur GRUB** sur le disque principal.

Une fois l’installation terminée, la machine redémarre sur Debian.

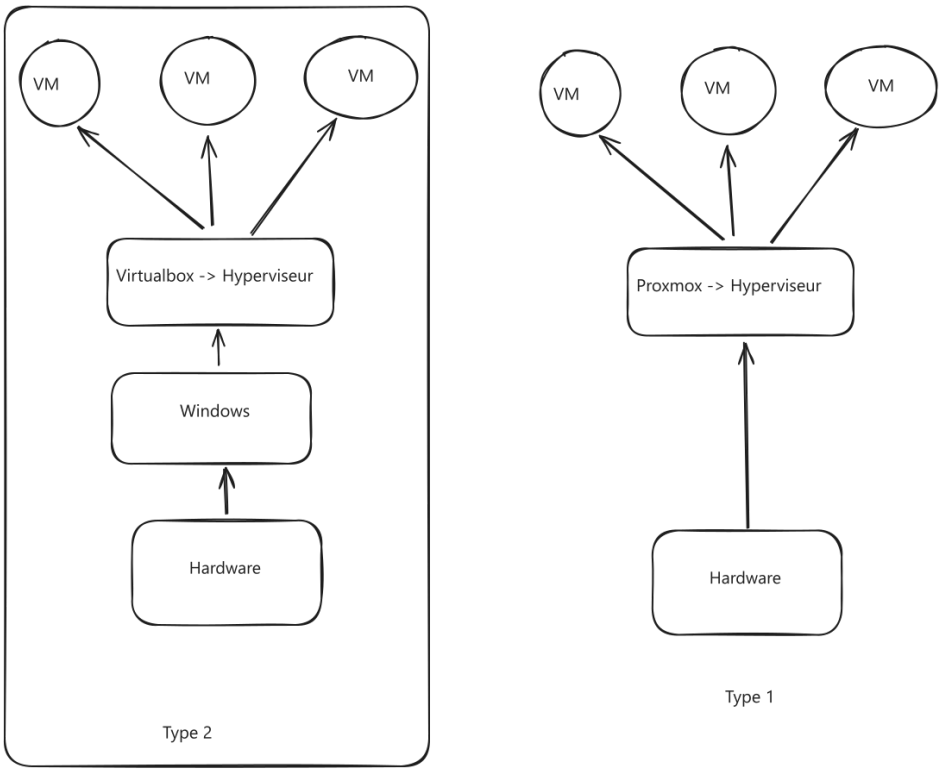
Configuration réseau de la VM

Type d’adaptateur	Description	Utilisation
NAT	La VM accède à Internet via la machine hôte.	Téléchargements, mises à jour.
Host-Only	Réseau interne entre l’hôte et la VM.	Tests et échanges de fichiers locaux.
Bridged (pont)	La VM est visible sur le réseau local.	Cas avancés ou tests réseau.

Hôte, invité et hyperviseur

L’**hôte** est l’ordinateur physique et son système ; l’**invité** est le système installé dans la VM. L’hyperviseur partage les ressources du matériel entre les invités.

Un hyperviseur de **type 1** s'exécute directement sur le matériel ; un **type 2**, comme VirtualBox dans ce cours, fonctionne au-dessus d'un système hôte. Hyper-V utilise une architecture de type 1 même sur un poste Windows.



DR02.Hyperviseur-Type

Instantané et clone

Un **instantané** aide à revenir à un état précédent avant une manipulation. Un **clone** crée une autre VM. Un instantané stocké avec le disque virtuel ne remplace pas une sauvegarde indépendante.

Après un clone destiné à devenir une nouvelle machine, vérifier le nom d'hôte, l'adresse MAC, l'adressage IP, l'identité système et les clés d'hôte SSH. Les outils de clonage et de préparation d'images peuvent prendre en charge une partie de cette régénération.

VM, conteneur et cloud

Élément	Idée à retenir
VM	Possède son propre noyau et ses disques virtuels.
Conteneur Linux	Partage le noyau de l'hôte Linux ; isole des processus et leur environnement.
Instance IaaS	Machine fournie à la demande avec réseau et volumes de stockage.
Image cloud	Modèle de système prêt à être instancié, souvent initialisé par Cloud-init.

Pour la formation, conserver une VM Debian comme environnement commun. Le complément Cloud-init montre ensuite un exemple d'initialisation automatisée.

À retenir

- Créer une **VM VirtualBox** permet d'installer Linux sans modifier son PC. - Allouer environ **2–4 Go de RAM** et **20–40 Go de disque**. - Utiliser un réseau **NAT** pour Internet et **Host-Only** pour le lien local. - Suivre les étapes de l'installateur Debian : langue, réseau, utilisateurs, partitions, paquets. - Installer **GRUB** pour permettre le démarrage du système.

06. Accéder à la VM et préparer sa session

Objectif Ouvrir une session locale ou distante et préparer les outils nécessaires.

Accéder à la machine virtuelle

Une fois Debian installée, il existe plusieurs manières d'interagir avec la machine virtuelle. Le choix dépend du **confort souhaité** et du **type d'utilisation** (formation, serveur, tests, etc.).

Accès direct via VirtualBox

L'accès le plus simple consiste à **ouvrir la console intégrée** de VirtualBox. C'est l'écran affiché par défaut au démarrage de la VM.

Améliorer l'expérience : *les "Guest Additions"*

Les **additions invité** permettent :

- Le **redimensionnement automatique** de la fenêtre,
- Le **partage du presse-papiers**,
- Le **glisser-déposer** entre hôte et VM,
- Le **partage de dossiers**.

Documentation officielle :

- Guest Additions VirtualBox (Oracle)
- Installer les Guest Additions sur Linux

Première connexion SSH

SSH (Secure Shell) permet de se connecter à un système Linux depuis un autre ordinateur, **en ligne de commande**. Il suffit que le paquet **openssh-server** soit installé sur la VM.

```
sudo apt install openssh-server
```

Connexion depuis une autre machine :

```
ssh utilisateur@adresse_ip
```

Outils d’accès SSH

Outil	Plateforme	Points forts	Limites
Windows Terminal	Windows 10/11	Intégré, rapide, moderne, compatible avec WSL	Interface uniquement en ligne de commande
Visual Studio Code (Remote SSH)	Multiplateforme	Accès SSH avec interface graphique et éditeur complet	Nécessite l’extension *Remote - SSH*
MobaXterm	Windows	Interface riche, gestion de sessions multiples, transfert de fichiers intégré	Logiciel tiers, non libre

---

Un environnement agréable

Disposer d’un **environnement de travail clair et personnalisé** est essentiel pour travailler efficacement :

- Affichage lisible, terminal coloré, invite de commande adaptée ;
- Outils essentiels installés (`vim`, `htop`, `curl`, `tree`, etc.) ;
- Accès fluide à la VM via la méthode de ton choix.

Un environnement bien configuré améliore la **productivité**, la **concentration** et le **plaisir d’utilisation**.

Préparer les outils de formation

```
sudo apt update
sudo apt install sudo nano vim less man-db curl htop
```

Si `sudo` n’est pas encore installé ou si le compte n’est pas autorisé, utiliser la console de la VM et `su -` avec le mot de passe root défini à l’installation. Depuis cette session root :

```
apt update
apt install sudo
usermod -aG sudo sam
exit
```

Remplacer `sam` par le compte créé pendant l'installation. Fermer puis rouvrir sa session pour que l'ajout au groupe soit pris en compte. Le chapitre sur les utilisateurs explique les groupes ; celui sur APT détaille les paquets.

Reconnaître sa session

```
whoami
hostname
pwd
```

Le premier résultat indique le compte, le deuxième la machine et le troisième le répertoire courant. C'est utile avant toute modification de configuration.

À retenir

- On peut accéder à une machine virtuelle **directement depuis VirtualBox** grâce à la console intégrée.

- L'installation des **Guest Additions** améliore le confort (redimensionnement, presse-papiers, partage de fichiers).
- L'accès **SSH** permet de se connecter à distance en ligne de commande.
- Plusieurs outils existent pour le SSH :
- **Windows Terminal** : rapide et intégré, mais sans interface graphique.
- **VS Code (Remote SSH)** : offre un environnement complet et visuel.
- **MobaXterm** : pratique pour les transferts de fichiers, mais non libre.
- Un **environnement de travail agréable** (couleurs, outils, confort d'accès) favorise la productivité et l'apprentissage.

07. Comprendre et utiliser une commande

Objectif Lire une commande, identifier ses options et consulter son aide.

Repères LPI 101 : 103.1.

Commandes à connaître : `man`, `help`, `echo`, `whoami`.

Introduction

Sous Linux, la majorité des opérations peuvent être effectuées en ligne de commande à l'aide du terminal. Le terminal est une interface textuelle qui permet d'interagir directement avec le système d'exploitation en tapant des commandes.

Ce chapitre explique la structure d'une commande et l'utilisation de son aide. Les deux suivants présentent la navigation et la lecture de fichiers.

Structure d'une commande

Une commande Linux suit en général la structure suivante :

```
commande [options] [arguments]
```

- **commande** : nom du programme ou de l'outil à exécuter (ex. `ls`, `cd`, `cat`).
- **options** : modifient le comportement de la commande (ex. `-l`, `-a`).
- **arguments** : précisent sur quoi la commande agit (fichiers, dossiers, texte, etc.).

Exemple :

```
ls -l /home
```

- `ls` : liste les fichiers.
- `-l` : affiche les détails (taille, droits, propriétaire…).
- `/home` : indique le répertoire sur lequel agir.

De nombreuses commandes disposent d'une page d'aide appelée **page de manuel** :

```
man ls
```

ou plus rapidement :

```
ls --help
```

Trouver de l'aide et quitter un programme

Dans `man ls`, utiliser `/mot` pour rechercher, `n` pour l'occurrence suivante et `q` pour quitter. Le numéro de section distingue les manuels : `man 1 passwd` décrit la commande ; `man 5 passwd` décrit le fichier.

```
man ls
man -k archive
ls --help
```

Geste	Utilité
Tab	Compléter un nom de commande ou un chemin ; deux pressions peuvent afficher les possibilités.
Flèches haut/bas	Reprendre les commandes précédentes.
Ctrl+C	Demander l'interruption du programme au premier plan.
Ctrl+L	Nettoyer l'affichage du terminal.
`q`	Quitter `less`, `man` ou `top`.

Les symboles `$` et `#` qui précèdent parfois les commandes dans une documentation représentent l'invite : ne pas les recopier. En général, `$` indique un compte standard et `#` une session root.

À retenir

Lire `commande options arguments`, consulter l'aide et savoir quitter le programme évitent beaucoup de blocages.

08. Se déplacer et comprendre les chemins

Objectif Utiliser les chemins absolus et relatifs sans confondre racine et dossier personnel.

Repères LPI 101 : 103.1, 104.7.

Commandes à connaître : `pwd`, `cd`, `ls`.

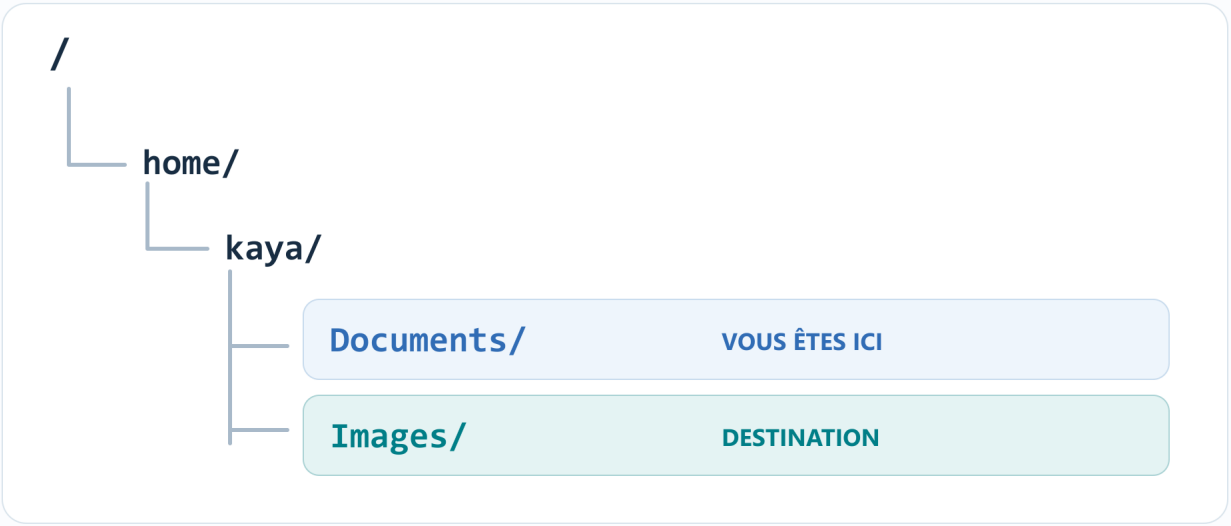
Navigation dans l’arborescence

Sous Linux, tout est organisé dans une **arborescence unique** partant de la racine `/`.

LPI 101 / REPÈRE VISUEL

Deux chemins, une même destination

On se trouve dans /home/kaya/Documents et on veut aller dans Images.



CHEMIN ABSOLU

```
cd /home/kaya/Images
```

CHEMIN RELATIF

```
cd ../Images
```

/ commence à la racine. .. remonte d'un niveau depuis le dossier courant.

Le chemin relatif dépend du point de départ ; le chemin absolu reste le même.

Chemin absolu et chemin relatif vers le même répertoire.

Les principales commandes de navigation sont :

`pwd` — afficher le répertoire courant

Affiche le chemin complet du répertoire dans lequel on se trouve.

```
pwd
```

Exemple de sortie :

```
/home/kaya/Documents
```

`cd` — changer de répertoire

Permet de se déplacer dans l'arborescence.

Exemples :

```
cd /home           # aller dans /home
cd ..              # remonter d'un dossier
cd                 # revenir dans le dossier personnel
cd ~/Documents     # aller directement dans Documents
```

`ls` — lister le contenu d'un répertoire

Affiche la liste des fichiers et dossiers du répertoire courant ou d'un autre répertoire.

Exemples :

```
ls                 # liste simple
ls -l              # liste détaillée
ls -a              # affiche les fichiers cachés
ls -lh /etc        # affiche les fichiers de /etc avec tailles lisibles
```

Quelques options utiles :

- `-l` : format long (permissions, propriétaire, taille…)
- `-a` : affiche aussi les fichiers cachés
- `-h` : affiche les tailles dans un format lisible (Ko, Mo)

Chemin absolu ou relatif

Un **chemin absolu** commence par `/` et part de la racine. Un **chemin relatif** part du répertoire courant.

```
/
├── etc/
└── home/
    └── sam/
        └── Documents/
```

Depuis `/home/sam`, `Documents` et `/home/sam/Documents` désignent le même dossier. `.` désigne le dossier courant, `..` son parent et `~` le dossier personnel du compte courant. `/root` est le dossier personnel de root ; `/` est la racine du système.

```
cd /home/sam
cd Documents
pwd
cd ..
cd -
```

`cd -` retourne au répertoire précédent. Adapter le nom du compte à la VM.

Casse, espaces et fichiers cachés

`rapport.txt` et `Rapport.txt` sont généralement deux noms différents sous Linux. Un nom commençant par `.` est caché dans l'affichage habituel de `ls`.

```
mkdir -p "$HOME/mes documents"
cd "$HOME/mes documents"
ls -a
```

Les guillemets gardent le chemin contenant un espace dans **un seul argument**. Dans les guillemets doubles, `$HOME` est développé ; dans les guillemets simples, il reste du texte littéral.

À retenir

Avant d'agir sur un fichier, vérifier le répertoire courant avec `pwd`, puis lire le chemin utilisé par la commande.

09. Lire et identifier des fichiers

Objectif Choisir un outil de lecture et reconnaître le type d'un fichier.

Repères LPI 101 : 103.2, 103.3.

Commandes à connaître : `cat`, `less`, `head`, `tail`, `file`, `stat`.

Consultation de fichiers

`cat` — afficher le contenu complet d'un fichier

```
cat /etc/hostname
```

Cette commande affiche tout le contenu du fichier à l'écran. Utile pour de petits fichiers, mais peu pratique pour les gros volumes de texte.

`less` — lecture page par page

```
less /var/log/syslog
```

Permet de naviguer dans le contenu avec les flèches du clavier. Touches utiles :

- `espace` → page suivante
- `b` → page précédente
- `/mot` → recherche d'un mot
- `q` → quitter

`head` — afficher les premières lignes d'un fichier

```
head -n 10 /etc/passwd
```

Affiche les 10 premières lignes du fichier (par défaut, 10 si `-n` non précisé).

`tail` — afficher les dernières lignes d'un fichier

```
tail -n 20 /var/log/syslog
```

Affiche les 20 dernières lignes du fichier. Option utile :

```
tail -f /var/log/syslog
```

`-f` permet de suivre en temps réel les ajouts au fichier (utile pour surveiller des logs).

Reconnaître le type d’un fichier

L’extension aide à se repérer, mais ne suffit pas à connaître le contenu réel.

```
file /etc/hostname
stat /etc/hostname
```

`file` tente d’identifier la nature du contenu. `stat` affiche notamment la taille, les permissions, les dates et le numéro d’inode. Les liens et les inodes sont expliqués dans leur chapitre.

À retenir

`cat` convient à un petit fichier ; `less` à une lecture longue ; `head` et `tail` à un extrait. `tail -f` attend les nouvelles lignes jusqu’à Ctrl+C.

10. Créer, copier, déplacer et supprimer

Objectif Manipuler les fichiers et répertoires en comprenant les effets des commandes.

Repères LPI 101 : 103.3.

Commandes à connaître : `touch`, `mkdir`, `rmdir`, `cp`, `mv`, `rm`.

Introduction

Sous Linux, la manipulation de fichiers et de répertoires se fait principalement à l'aide de la ligne de commande. Les commandes que nous allons voir permettent de **créer, supprimer, déplacer, copier et rechercher** des fichiers ou des dossiers. Elles constituent la base du travail quotidien sur un système Linux.

Création et suppression de fichiers et de répertoires

`touch` — créer un fichier vide

La commande `touch` permet de créer un fichier vide ou de mettre à jour la date de dernière modification d'un fichier existant.

```
touch fichier.txt
```

Si le fichier `fichier.txt` n'existe pas, il sera créé. S'il existe déjà, sa date de modification sera actualisée.

`mkdir` — créer un répertoire

```
mkdir dossier1
```

Crée un répertoire nommé `dossier1`. Pour créer plusieurs niveaux de répertoires d'un coup, utilisez l'option `-p` :

```
mkdir -p dossier1/sousdossier1/sousdossier2
```

`rmdir` — supprimer un répertoire vide

```
rmdir dossier1
```

Supprime un répertoire **vide uniquement**. Si le répertoire contient des fichiers, la commande échouera.

`rm` — supprimer un ou plusieurs fichiers ou répertoires

```
rm fichier.txt
```

Supprime le fichier spécifié. Pour supprimer un répertoire et tout son contenu, utilisez :

```
rm -r dossier1
```

L'option `-r` signifie "récursif". On peut ajouter `-f` pour forcer la suppression sans demande de confirmation :

```
rm -rf dossier1
```

□□ Cette commande est **destructive** : les fichiers supprimés de cette façon ne sont pas récupérables via une corbeille.

Déplacement et copie

`mv` — déplacer ou renommer un fichier

```
mv fichier.txt /home/kaya/Documents/
```

Déplace le fichier `fichier.txt` dans le répertoire `/home/kaya/Documents`. La même commande sert à renommer un fichier :

```
mv fichier.txt nouveau_nom.txt
```

Si la destination est un répertoire, le fichier est déplacé. Si la destination est un nom de fichier, il est renommé.

`cp` — copier un fichier ou un répertoire

```
cp fichier.txt /home/kaya/Documents/
```

Copie le fichier vers un autre répertoire. Pour copier un répertoire complet avec tout son contenu, ajoutez `-r` :

```
cp -r dossier1 sauvegarde_dossier1
```

L'option `-i` permet d'éviter d'écraser un fichier existant sans confirmation.

```
cp -i fichier.txt /tmp/
```

Distinguer les opérations

Commande	Résultat
`cp source copie`	Crée un autre fichier avec son propre contenu.
`mv ancien nouveau`	Déplace ou renomme l'entrée.
`rm fichier`	Supprime le nom du fichier ; aucune corbeille n'est utilisée par défaut.

```
mkdir -p ~/labo-lpi/documents
touch ~/labo-lpi/documents/notes.txt
cp -i ~/labo-lpi/documents/notes.txt ~/labo-lpi/documents/copie.txt
mv -i ~/labo-lpi/documents/copie.txt ~/labo-lpi/documents/rapport.txt
```

L'option `-i` demande confirmation avant certains écrasements. Pour copier un dossier avec son contenu : `cp -r dossier copie`. `cp -a` préserve davantage de métadonnées et les liens ; le résultat dépend aussi des droits de l'utilisateur et du système de fichiers.

À retenir

Vérifier le chemin avant une suppression, surtout avec `-r`. Pour apprendre, utiliser un dossier de travail dans son répertoire personnel.

11. Utiliser les jokers et rechercher

Objectif Distinguer les motifs de noms, la recherche de fichiers et la recherche de commandes.

Repères LPI 101 : 103.3, 104.7.

Commandes à connaître : `find`, `locate`, `updatedb`, `type`, `command -v`, `which`, `whereis`.

Utilisation des jokers (wildcards)

Les jokers permettent de sélectionner plusieurs fichiers à la fois selon un motif.

Joker	Description	Exemple
`*`	remplace n'importe quelle suite de caractères	`ls *.txt` → liste tous les fichiers se terminant par `.txt`
`?`	remplace un seul caractère	`ls fichier?.txt` → `fichier1.txt`, `fichier2.txt`, etc.
`[ ]`	remplace un caractère parmi ceux indiqués	`ls fichier[12].txt` → `fichier1.txt` et `fichier2.txt`

Exemples pratiques :

```
rm *.log              # supprime tous les fichiers .log
cp *.txt /tmp/        # copie tous les fichiers .txt vers /tmp
ls fichier[0-9].txt   # affiche les fichiers dont le nom contient un chiffre de 0 à 9
```

Commandes de recherche

`find` — rechercher des fichiers dans l'arborescence

`find` parcourt l'arborescence à partir d'un répertoire donné et affiche les fichiers correspondant à certains critères.

```
find /home -name "fichier.txt"
```

Cherche un fichier nommé `fichier.txt` dans le répertoire `/home` et ses sous-dossiers.
Quelques options utiles :

- `-type f` : ne recherche que des fichiers.
- `-type d` : ne recherche que des répertoires.

- `-mtime -1` : fichiers modifiés dans les dernières 24h.
- `-size +10M` : fichiers de plus de 10 Mo.

Exemples :

```
find /etc -type f -name "*.conf"
find /home -type f -size +1M
```

`locate` — rechercher rapidement un fichier

`locate` recherche un fichier dans une base de données pré-indexée (mise à jour avec `updatedb`).

```
locate passwd
```

Cette commande est beaucoup plus rapide que `find`, mais elle dépend de la dernière mise à jour de l'index :

```
sudo updatedb
```

Chercher un fichier ou une commande

```
type cd
type ls
command -v bash
which bash
whereis bash
```

`type` indique comment le shell interprète un nom : commande interne, alias, fonction ou programme. `command -v` indique ce que le shell utiliserait ; `which` recherche habituellement un exécutable dans `PATH`. `whereis` recherche des binaires, sources et manuels dans des emplacements connus. La disponibilité de `which` et `locate` dépend des paquets installés.

Filtrer une recherche par date et agir sur les résultats

```
find "$HOME" -type f -name '*.txt' -mtime -7
find "$HOME/labo-lpi" -type f -name '*.txt' -exec wc -l {} +
```

Le premier exemple cherche des fichiers texte modifiés depuis moins de sept jours. Le second compte leurs lignes en transmettant les chemins à `wc`. Les guillemets empêchent le shell de développer `*.txt` avant que `find` le reçoive.

`-mtime` concerne la modification du contenu ; `-atime`, l'accès ; `-ctime`, le changement des métadonnées de l'inode. `ctime` n'est pas la date de création.

À retenir

`find` recherche dans l'arborescence ; `locate` consulte une base ; `type` renseigne sur une commande ; `grep` recherche dans du texte.

12. Comprendre les liens physiques et symboliques

Objectif Créer un lien et prévoir ce qui se passe lorsque sa cible est déplacée ou supprimée.

Repères LPI 101 : 104.6.

Commandes à connaître : `ln`, `ls -li`, `readlink`, `stat`.

Un fichier, un nom et un inode

Un fichier possède des métadonnées stockées dans un **inode**. Un répertoire associe un nom à un numéro d'inode. Les données sont stockées séparément des noms.

Un **lien physique** ajoute un autre nom pour le même inode sur le même système de fichiers. Il n'y a pas de fichier « original » privilégié : les noms donnent accès au même contenu.

```
mkdir -p ~/labo-liens
cd ~/labo-liens
printf 'Version 1\n' > original.txt
ln original.txt physique.txt
ls -li original.txt physique.txt
printf 'Version 2\n' >> physique.txt
cat original.txt
```

Les deux noms ont le même numéro d'inode. Modifier le contenu en place par l'un des noms le modifie pour l'autre. Certains éditeurs remplacent cependant le fichier à l'enregistrement, créant alors un nouvel inode.

Le lien symbolique

Un **lien symbolique** est un fichier distinct contenant un chemin vers sa cible. Il peut viser un dossier ou traverser les systèmes de fichiers.

```
ln -s original.txt raccourci.txt
ls -l raccourci.txt
readlink raccourci.txt
```

```
original.txt ─┐
              ├─ inode commun → contenu
physique.txt ─┘
raccourci.txt → chemin « original.txt »
```

Une cible relative est interprétée depuis le dossier contenant le lien symbolique, pas depuis le dossier courant de l'utilisateur.

Supprimer un nom

Dans ce dossier de démonstration :

```
rm original.txt
cat physique.txt
ls -l raccourci.txt
```

Le contenu reste accessible par `physique.txt`. Le lien symbolique devient **cassé**, car le chemin `original.txt` n'existe plus. Les données peuvent être libérées lorsque le dernier lien physique disparaît et qu'aucun processus ne conserve le fichier ouvert.

Copie ou lien ?

Opération	Inode distinct	Contenu indépendant	Traverse les systèmes de fichiers
Copie	Oui	Oui	Oui
Lien physique	Non	Non	Non
Lien symbolique	Oui, pour le lien	Renvoie vers une cible	Oui

Les liens physiques vers des répertoires ne sont pas autorisés dans l'usage courant. `cp -a` conserve les liens symboliques ; `cp -L` suit leurs cibles.

À retenir

`ln` ajoute un nom au même fichier ; `ln -s` crée un chemin vers une cible ; `cp` crée une copie indépendante.

13. Archiver et compresser

Objectif Distinguer regroupement de fichiers et réduction de leur taille.

Repères LPI 101 : 103.3.

Commandes à connaître : `tar`, `gzip`, `gunzip`, `bzip2`, `bunzip2`, `xz`, `unxz`.

Introduction

Une archive rassemble des fichiers ; la compression réduit leur taille. Ce chapitre présente `tar`, `gzip`, `bzip2` et `xz`. Les transferts SSH, la recherche de texte et les redirections sont expliqués dans leurs chapitres respectifs.

Création et extraction d'archives

`tar` — créer et extraire des archives

`tar` (Tape ARchive) regroupe plusieurs fichiers dans une seule archive. Elle ne compresse pas par défaut, mais on peut l'associer à `gzip` ou `bzip2`.

Créer une archive :

```
tar -cvf archive.tar dossier1
```

- `-c` : créer une archive
- `-v` : afficher les fichiers ajoutés (mode verbeux)
- `-f` : nom du fichier archive

Extraire une archive :

```
tar -xvf archive.tar
```

Lister le contenu sans extraire :

```
tar -tvf archive.tar
```

Compression avec `gzip`

`gzip` compresse un fichier unique (extension `.gz`).

```
gzip fichier.txt
```

Le fichier devient `fichier.txt.gz`. Pour le décompresser :

```
gunzip fichier.txt.gz
```

`tar` peut l'utiliser directement :

```
tar -czvf archive.tar.gz dossier1
tar -xzvf archive.tar.gz
```

Compression avec `bzip2`

`bzip2` offre souvent une compression plus forte que `gzip`, avec un temps de traitement plus long ; le résultat dépend des données (extension `.bz2`).

```
bzip2 fichier.txt
bunzip2 fichier.txt.bz2
```

Avec `tar` :

```
tar -cjvf archive.tar.bz2 dossier1
tar -xjvf archive.tar.bz2
```

Compression avec xz

```
xz -k rapport.txt
mkdir -p decompression
xz -dc rapport.txt.xz > decompression/rapport.txt
tar -cJf documents.tar.xz Documents/
tar -tJf documents.tar.xz
```

`-k` conserve le fichier d'entrée ; `tar -t` liste le contenu d'une archive. `unxz fichier.xz` décompresse normalement à côté du fichier compressé ; ici `xz -dc` écrit vers un autre dossier et garde le fichier `.xz`.

Vérifier avant d'extraire

```
tar -tf sauvegarde.tar
mkdir -p restauration
tar -xf sauvegarde.tar -C restauration
```

Adapter le nom à une archive créée précédemment. Lister le contenu permet de voir les chemins avant l'extraction. Les transferts réseau avec `scp` se trouvent dans le chapitre SSH.

À retenir

`tar` regroupe plusieurs fichiers ; `gzip`, `bzip2` et `xz` compressent. Une archive `.tar.gz` combine les deux opérations.

14. Variables, environnement et historique

Objectif Comprendre où Bash cherche les commandes et réutiliser son historique.

Repères LPI 101 : 103.1.

Commandes à connaître : `echo`, `env`, `export`, `set`, `unset`, `history`, `type`, `command -v`.

Fichiers et emplacements : `~/.bash_history`, `~/.bashrc`.

Afficher une valeur

```
echo "$HOME"
echo "$USER"
echo "$PWD"
echo "$PATH"
```

`HOME` désigne le dossier personnel, `USER` un nom d'utilisateur fourni par l'environnement et `PWD` le répertoire courant du shell. `whoami` indique l'identité effective de la commande.

`PATH` est une liste de dossiers séparés par `:`. Bash y cherche les programmes dont on saisit le nom sans chemin. Pour un script présent dans le dossier courant, on utilise généralement `./script.sh`.

Variable du shell et variable exportée

```
message="Bonjour Sam"
echo "$message"
export message
bash -c 'echo "$message"'
unset message
```

L'affectation ne contient aucun espace autour de `=`. `export` transmet la variable aux futurs processus enfants. Une modification dans un enfant ne change pas la variable de son parent. `env` affiche l'environnement exporté ; `set` affiche aussi des variables et fonctions du shell et produit souvent une longue sortie.

Comprendre les guillemets et les substitutions

```
nom="Sam"
echo "Bonjour $nom"
echo 'Bonjour $nom'
aujourdhui=$(date +%F)
echo "$aujourdhui"
```

Les guillemets doubles permettent l'expansion de `$nom` tout en gardant un argument. Les simples conservent le texte littéral. `$(commande)` remplace l'expression par la sortie de la commande.

Retrouver une commande

```
history
history 10
```

Utiliser les flèches haut/bas pour parcourir les dernières commandes et **Ctrl+R** pour rechercher un fragment. Modifier la commande retrouvée avant de l'exécuter. L'historique est généralement enregistré dans `~/.bash_history`, avec un comportement qui dépend de la configuration et des sessions ouvertes.

Ajouter un dossier de commandes

```
mkdir -p "$HOME/bin"
export PATH="$HOME/bin:$PATH"
command -v bash
```

Cette modification concerne la session et ses futurs enfants. Le complément sur `.bashrc` explique comment conserver une personnalisation. Ne pas supprimer les anciennes valeurs de `PATH`.

À retenir

`$nom` lit une variable, `export` la transmet et `PATH` sert à retrouver les programmes. L'historique évite de tout retaper.

15. Flux, redirections et tubes

Objectif Relier des commandes et distinguer résultat normal et message d'erreur.

Repères LPI 101 : 103.4.

Commandes à connaître : `echo`, `tee`, `xargs`.

Introduction

Sous Linux, **tout est flux** : chaque programme communique via des canaux standards d'entrée et de sortie. Comprendre et manipuler ces flux est essentiel pour combiner les commandes, automatiser des traitements et chaîner des opérations complexes.

Les trois flux standards

Nom	Numéro	Description	Exemple
stdin	0	Entrée standard — données lues par le programme	`cat < fichier.txt`
stdout	1	Sortie standard — résultat "normal" d'un programme	`echo "OK" > fichier.txt`
stderr	2	Sortie d'erreur — messages d'erreur séparés du résultat	`ls /inexistant 2> erreur.log`

Redirections de base

• **Envoyer la sortie vers un fichier (création/écrasement)**

```
bash commande > fichier.txt
```

Remplace le contenu du fichier par la sortie.

• **Ajouter à la fin d'un fichier (sans écraser)**

```
bash commande >> fichier.txt
```

Concatène le résultat au fichier existant.

• **Lire à partir d'un fichier**

```
bash commande < fichier.txt
```

Fournit le contenu du fichier comme entrée.

Redirection des erreurs

• **Rediriger les erreurs vers un fichier dédié**

```
bash commande 2> erreurs.log
```

• **Rediriger la sortie et les erreurs ensemble**

```
bash commande &> tout.log
```

ou équivalent POSIX :

```
bash commande > tout.log 2>&1
```

• **Ignorer les erreurs (envoyer vers le "trou noir")**

```
bash commande 2>/dev/null
```

Chaînage de commandes

• **Enchaîner seulement si la commande précédente réussit**

```
bash commande1 && commande2
```

(`commande2` ne s'exécute que si la première renvoie un code 0)

• **Enchaîner seulement si la commande précédente échoue**

```
bash commande1 || commande2
```

(`commande2` ne s'exécute que si la première échoue)

• **Exécuter plusieurs commandes à la suite, peu importe le résultat**

```
bash commande1 ; commande2
```

Les pipes (`|`)

Le **pipe** relie la sortie standard d'une commande à l'entrée standard d'une autre :

```
commande1 | commande2
```

Exemples :

```
ls -l /etc | less
cat /etc/passwd | grep root
```

Le tube permet de chaîner les commandes sans passer par des fichiers temporaires : on crée ainsi des **chaînes de traitement efficaces**.

Codes de retour

Chaque commande se termine par un **code de sortie** :

- `0` → succès
- autre → erreur

Afficher le dernier code de retour :

```
echo $?
```

Utile dans les scripts ou pour tester le comportement des commandes combinées avec `&&` et `||`.

Afficher et enregistrer avec tee

```
printf 'Bonjour\n' | tee resultat.txt
printf 'Suite\n' | tee -a resultat.txt
```

`tee` envoie les mêmes données vers l'écran et le fichier ; `-a` ajoute au lieu d'écraser. Il n'est donc plus nécessaire de relancer la commande pour voir son résultat.

Transmettre des arguments avec xargs

```
printf '%s\0' /etc/hostname /etc/hosts | xargs -0 wc -l
```

Le tube transmet du texte sur l'entrée standard ; `xargs` transforme cette entrée en **arguments** de la commande `wc`. Le séparateur nul et `-0` permettent de conserver les noms contenant des espaces.

Lire l'ordre des redirections

```
ls /etc /inexistant > resultat.log 2>&1
```

La sortie standard va d'abord dans le fichier ; la sortie d'erreur est ensuite reliée à cette destination. Inverser `2>&1` et `> resultat.log` change le résultat.

À retenir

* Linux gère trois flux : **entrée (0)**, **sortie (1)** et **erreur (2)**. * Les redirections `>`, `>>`, `<` permettent d'écrire ou de lire depuis un fichier. * `2>` redirige les erreurs ; `&>` combine sortie + erreur. * Le symbole `|` (pipe) envoie la sortie d'une commande vers une autre. * Les opérateurs `&&`, `||`, `;` permettent de contrôler l'exécution selon le résultat. * `$?` affiche le code de retour de la dernière commande.

16. Filtrer et transformer du texte

> Objectif Rechercher, extraire, trier, compter et transformer des lignes.

Repères LPI 101 : 103.2.

Commandes à connaître : `grep`, `cut`, `sort`, `uniq`, `wc`, `tr`, `sed`, `awk`.

Introduction

Les **filtres** sont des commandes qui lisent un flux de texte (souvent depuis l'entrée standard) et produisent un nouveau flux en sortie. Ils sont souvent utilisés avec les **pipes (|)** pour transformer, trier ou extraire des informations à partir de fichiers ou de commandes.

Commandes de base

`cat` — afficher le contenu d'un fichier

`cat`, `head` et `tail` ont été présentés dans le chapitre 09. On les utilise ici comme étapes d'une chaîne de traitement ; `cat` peut aussi concaténer plusieurs fichiers.

`grep` — rechercher un motif dans un texte

```
grep motif fichier.txt
```

Exemples :

```
grep root /etc/passwd
grep -i "sam" fichier.txt      # ignore la casse
grep -v "daemon" fichier.txt   # inverse la recherche (exclut)
```

Options utiles :

- `-n` : affiche les numéros de ligne
- `-r` : recherche récursive dans un répertoire
- `-E` : active les expressions régulières étendues

`cut` — extraire des colonnes ou des champs

```
cut -d ':' -f 1 /etc/passwd
```

Options principales :

- `-d` : définit le délimiteur (ici `:`)
- `-f` : sélectionne les champs à afficher

Exemple : extraire les noms d'utilisateur du fichier `/etc/passwd`.

`sort` — trier les lignes d'un texte

```
sort fichier.txt
```

Options :

- `-r` : ordre inverse
- `-n` : tri numérique
- `-u` : supprime les doublons

Exemples :

```
sort -u noms.txt             # tri alphabétique unique
sort -nr nombres.txt         # tri numérique inverse
```

`uniq` — supprimer les doublons consécutifs

```
uniq fichier.txt
```

Souvent utilisé après `sort`, car `uniq` ne détecte que les doublons **successifs**. Exemple :

```
sort noms.txt | uniq
```

Options :

- `-c` : affiche le nombre d'occurrences de chaque ligne

`wc` — compter lignes, mots, caractères

```
wc fichier.txt
```

Affiche trois valeurs :

1. nombre de lignes
2. nombre de mots
3. nombre d'octets

Exemples :

```
wc -l fichier.txt    # lignes seulement
wc -w fichier.txt    # mots seulement
wc -c fichier.txt    # octets
wc -m fichier.txt    # caractères
```

`tr` — transformer ou supprimer des caractères

```
tr a-z A-Z < fichier.txt
```

→ convertit le texte en majuscules.

Autres exemples :

```
tr -d ' ' < fichier.txt        # supprime les espaces
tr ':' ',' < fichier.txt       # remplace les deux-points par des virgules
```

`head` et `tail` — afficher le début ou la fin d'un fichier

Dans un tube, `head -n 10` conserve les dix premières lignes ; `tail -n 20`, les vingt dernières. Voir le chapitre 09 pour la lecture et le suivi d'un fichier.

Chaînage des filtres

Les filtres deviennent puissants lorsqu'on les combine :

```
cat /etc/passwd | cut -d ':' -f 1 | sort | uniq | wc -l
```

→ compte les noms de comptes distincts du fichier local `/etc/passwd`.

Autres exemples :

```
dmesg | grep error | tail -n 5
ls /bin | grep ssh | sort
```

`sed` — modifier le texte à la volée

`sed` (Stream EDitor) est un éditeur de flux : il lit ligne par ligne et applique des transformations sans ouvrir le fichier.

Exemples :

```
sed 's/root/admin/' /etc/passwd         # remplace le premier "root" par "admin" dans
    chaque ligne
sed 's/root/admin/g' /etc/passwd        # remplace toutes les occurrences sur chaque ligne
sed -n '1,5p' fichier.txt               # affiche uniquement les lignes 1 à 5
sed '/^#/d' fichier.txt                 # supprime les lignes commençant par #
```

`sed` est idéal pour automatiser de petites modifications dans des scripts ou pipelines.

`awk` — traiter et filtrer des colonnes

`awk` lit les fichiers ligne par ligne et découpe chaque ligne en **champs** séparés (par défaut par des espaces). C'est un outil très puissant pour l'analyse de texte et de rapports.

Exemples :

```
awk -F: '{print $1}' /etc/passwd        # affiche le 1er champ (nom d'utilisateur)
awk '{print $1, $3}' fichier.txt        # affiche les colonnes 1 et 3
awk '{s+=$1} END {print s}' data.txt    # additionne tous les nombres de la première
    colonne
awk '/root/ {print $1,$6}' /etc/passwd  # affiche certaines colonnes pour les lignes
    contenant "root"
```

`awk` est souvent utilisé pour extraire ou reformater des données tabulaires.

À retenir

* Les **filtres** transforment un flux de texte en entrée pour produire un résultat en sortie. * Les plus courants sont : `grep`, `cut`, `sort`, `uniq`, `wc`, `tr`, `head`, `tail`. * Les **pipes** (`|`) permettent de combiner plusieurs filtres pour créer une chaîne de traitement. * `grep` recherche, `cut` extrait, `sort` trie, `uniq` élimine les doublons, `wc` compte, `tr` remplace, `head/tail` affichent une partie. * Les exemples affichent leur résultat sans modifier le fichier d'entrée ; certaines options, comme `sed -i`, peuvent modifier un fichier. * `sed` permet d'éditer ou de remplacer du texte à la volée. * `awk` permet de traiter des colonnes et d'appliquer des conditions sur les lignes.

17. Rechercher avec les expressions régulières

Objectif Construire des motifs simples et distinguer expressions régulières et jokers du shell.

Repères LPI 101 : 103.7.

Commandes à connaître : `grep`, `grep -E`, `sed`.

Avant tout : ne pas confondre wildcards et expressions régulières

Les **wildcards** (ou *motifs génériques du shell*) et les **expressions régulières** (ou *regex*) se ressemblent, mais ce ne sont **pas du tout la même chose**.

Type	Utilisation	Exemple	Signification
Wildcards	Utilisés par le **shell** (ex : `ls`, `rm`, `cp`)	`ls *.txt`	Tous les fichiers qui se terminent par `.txt`
Regex	Utilisées par des outils comme `grep`, `sed`, `awk`, `find`, etc.	`grep '.*txt$' fichier`	Toutes les lignes qui se terminent par "txt"

Différences clés :

- Les **wildcards** servent à filtrer des **noms de fichiers**, pas du texte.
- Les **regex** servent à chercher des **motifs dans du contenu texte** (lignes de fichiers, logs, etc.).
- Les **symboles n'ont pas la même signification** :
- En shell : `*` = une suite de zéro ou plusieurs caractères
- En regex : `*` = "répète le caractère précédent 0 ou plusieurs fois"

Introduction

Les **expressions régulières** (ou **regex**) sont des motifs de recherche puissants permettant d'analyser, filtrer et manipuler du texte de manière avancée. Elles sont utilisées dans de nombreux outils Linux (`grep`, `sed`, `awk`, `find`, `vim`, etc.) et dans la plupart des langages de programmation.

Notions de base

Une expression régulière décrit **un motif de caractères** à rechercher dans un texte. Elle permet d'aller bien au-delà d'une simple recherche de mot exact.

Exemples simples :

Motif	Signification	Exemple de correspondance
`root`	Mot exact	"root"
`r..t`	`.` remplace n'importe quel caractère	"root", "rebt"
`^user`	Commence par "user"	"username"
`bash$`	Finit par "bash"	"/bin/bash"
`[0-9]`	Un chiffre entre 0 et 9	"3", "8"
`[A-Za-z]`	Une lettre majuscule ou minuscule	"a", "Z"
`[^0-9]`	Tout sauf un chiffre	"a", "-"
`*`	Répétition (0 ou plusieurs fois)	`go*` → "g", "go", "goo"
`+`	Répétition (1 ou plusieurs fois)	`go+` → "go", "goo"
`?`	Optionnel (0 ou 1 fois)	`colou?r` → "color" ou "colour"

Types d'expressions régulières

Sous Linux, il existe deux types principaux de syntaxe pour les expressions régulières POSIX :

BRE (Basic Regular Expressions)

→ utilisées par défaut avec la commande `grep`.

Certains symboles spéciaux doivent être **échappés avec un antislash (`\`)** pour être reconnus. C'est une syntaxe plus ancienne et un peu moins lisible.

Exemple :

```
grep '^\(root\|daemon\):' /etc/passwd
```

Explications :

- `\(` et `\)` → délimitent un **groupe** (il faut les échapper)
- `\|` → signifie **OU** avec GNU grep ; c'est une extension de sa syntaxe BRE
- `^` → début de ligne
- `:` → séparateur après le nom du compte (dans cet exemple)

Cette commande affiche toutes les **lignes dont le premier champ est "root" ou "daemon"** dans `/etc/passwd`.

ERE (Extended Regular Expressions)

→ activées avec `grep -E` (ancien nom : `egrep`).

Dans cette version, les symboles `()`, `|`, `+`, `?` et `{}` sont **interprétés directement**, sans besoin d'antislashs.

Exemple équivalent :

```
grep -E '^(root|daemon):' /etc/passwd
```

Explications :

- Les parenthèses `()` et le `|` sont compris **sans les** `\`, donc la syntaxe est **plus simple et plus lisible**.
- Le motif est identique en logique : il recherche les lignes dont le premier champ est **"root" ou "daemon"**.

Comparatif clair

Objectif	Syntaxe BRE (grep)	Syntaxe ERE (grep -E)
Grouper	`\( ... \)`	`( ... )`
OU logique	`\\|`	`|`
Un ou plusieurs (+)	`\+`	`+`
Zéro ou un (?)	`\?`	`?`
Répétition {m,n}	`\{m,n\}`	`{m,n}`

- `grep` (BRE) = plus ancien, **antislash obligatoire** pour certains symboles.

- `grep -E` (ERE) = **plus moderne et plus lisible**, plus courant aujourd'hui.
- Les deux font la même chose, mais **ERE est recommandé** pour plus de clarté.

Exemples pratiques

```
# Lignes commençant par root
grep '^root' /etc/passwd

# Lignes finissant par /bin/bash
grep '/bin/bash$' /etc/passwd

# Lignes contenant un chiffre
grep '[0-9]' /etc/passwd

# Lignes ne contenant PAS de chiffre
grep -v '[0-9]' /etc/passwd

# Utilisation d'une expression étendue (option -E)
grep -E '^(root|daemon)' /etc/passwd
```

Combinaisons avec sed et awk

- Avec `sed` :

`bash sed -n '/bash$/p' /etc/passwd sed 's/[0-9]/#/g' fichier.txt # remplace les chiffres par #`

- Avec `awk` :

`bash awk -F: '/bash$/ {print $1}' /etc/passwd awk -F: '/^root:/ {print $1,$6}' /etc/passwd`

Outils interactifs pour s'entraîner

- Regex101 — permet de tester ses expressions avec explications en direct (supporte plusieurs syntaxes).
- RegexOne — propose des **leçons progressives** et des exercices interactifs pour apprendre étape par étape.
- Regexlearn — offre un **apprentissage interactif et progressif en français**, idéal pour s'initier pas à pas aux expressions régulières.

À retenir

* Les **expressions régulières** servent à rechercher, filtrer et manipuler du texte de manière très fine. * Les symboles principaux :

- `.` : un caractère quelconque
- `^` : début de ligne
- `$` : fin de ligne
- `[]` : ensemble de caractères autorisés
- `[^]` : ensemble interdit
- `*`, `+`, `?` : répétitions
- `|` : OU logique
- `()` : regroupement
- **BRE** (basic) pour `grep`, **ERE** (extended) pour `grep -E` (ancien nom : `egrep`).
- Outils principaux : `grep`, `sed`, `awk`.
- Pour s'exercer :
- Regex101.com
- RegexOne.com
- Regexlearn

18. Modifier un fichier avec nano et vi

Objectif Ouvrir, modifier et enregistrer un fichier texte depuis le terminal.

Repères LPI 101 : 103.8.

Commandes à connaître : `nano`, `vi`, `vim`, `vimtutor`.

Rôle des éditeurs de texte

Sous Linux, la majorité des fichiers de configuration sont **des fichiers texte**. Il est donc essentiel de savoir les **ouvrir, modifier et enregistrer** en ligne de commande.

Les éditeurs de texte permettent d'éditer ces fichiers directement depuis le terminal, sans interface graphique.

Les principaux éditeurs de texte

Éditeur	Type	Caractéristiques principales	Utilisation recommandée
nano	Simple	Interface claire, commandes affichées en bas de l'écran	Débutants
vim / vi	Avancé	Très puissant, modes multiples (commande / insertion)	Administrateurs expérimentés
micro *(optionnel)*	Moderne	Coloration syntaxique, gestion de la souris, intuitive	Alternative pratique
gedit / mousepad	Graphique	Interface graphique simple (si environnement GNOME/XFCE)	Utilisation bureautique

---

Nano — l'éditeur le plus simple

Nano est souvent installé par défaut. Pour éditer un fichier :

```
nano /etc/hosts
```

Commandes utiles :

Action	Touche
Enregistrer	`Ctrl + O`
Quitter	`Ctrl + X`
Rechercher	`Ctrl + W`
Aller à une ligne	`Ctrl + _`

En bas de l'écran, le symbole `^` signifie **Ctrl**. Exemple : `^X` = `Ctrl + X`.

Vim — l'éditeur puissant

Vim (ou son ancêtre vi) est plus complexe, mais indispensable à connaître pour l'examen LPI et les environnements serveur.

Pour l'ouvrir :

```
vim /etc/hosts
```

Les deux modes principaux :

Mode	Description	Entrée / sortie
Mode commande	Navigation, suppression, recherche, enregistrement	`Échap` pour y revenir
Mode insertion	Édition du texte librement	`i` pour insérer du texte

Commandes essentielles :

Action	Commande
Passer en insertion	`i`
Sauvegarder	`:w`
Quitter	`:q`
Sauvegarder et quitter	`:wq`
Quitter sans enregistrer	`:q!`
Rechercher un mot	`/mot`

Pour débuter, tape :

```
vimtutor
```

C'est un tutoriel interactif intégré.

Micro — une alternative moderne

`micro` est un éditeur simple et coloré, inspiré de Sublime Text. Il n'est pas inclus par défaut mais peut être installé facilement :

```
sudo apt install micro
```

Caractéristiques :

- Navigation au clavier ou à la souris
- Coloration syntaxique
- Copie/collage classiques (`Ctrl + C / V`)
- Barre de statut intuitive

C'est un excellent compromis pour les utilisateurs débutants souhaitant un peu plus de confort que nano.

Éditeurs graphiques

Sur un environnement graphique, des éditeurs comme **gedit**, **mousepad** ou **kate** permettent de modifier les fichiers plus facilement, mais leur usage est limité aux systèmes avec interface graphique (non recommandé sur serveur).

Les gestes essentiels dans vi

Dans le mode normal, les lettres sont des commandes. Appuyer sur Échap pour revenir dans ce mode avant les actions suivantes.

Action	Touche ou commande
Déplacer le curseur	`h`, `j`, `k`, `l` ou les flèches
Début / fin de ligne	`0` / `$`
Début / fin de fichier	`gg` / `G`
Supprimer une ligne	`dd`
Copier une ligne	`yy`
Coller après le curseur	`p`
Annuler	`u`
Refaire	Ctrl+R
Occurrence suivante d'une recherche	`n`

Choisir son éditeur

```
export EDITOR=nano
export VISUAL=nano
```

Ces variables indiquent une préférence aux programmes qui les consultent. Sous Debian, `select-editor` configure aussi l'éditeur utilisé par certains outils de la distribution. Nano reste l'outil de départ ; les gestes vi constituent une petite compétence à acquérir progressivement.

À retenir

- De nombreux fichiers de configuration Linux sont **du texte brut**. - **Nano** est l'éditeur le plus simple pour les débutants. - **Vim** est un outil courant en administration Linux. - On peut installer **micro** pour plus de confort. - Savoir **quitter et sauvegarder** sous Vim est une compétence indispensable.

19. Utilisateurs, groupes et droits administrateur

> Objectif Reconnaître un compte et donner les droits nécessaires sans travailler constamment en root.

Commandes à connaître : `id`, `groups`, `useradd`, `usermod`, `passwd`, `groupadd`, `sudo`, `su`.

Introduction

Sous Linux, la gestion des utilisateurs et des groupes permet de **contrôler l'accès au système** et de **séparer les droits** entre les différents comptes. Chaque utilisateur dispose d'un identifiant unique, d'un répertoire personnel et d'un ensemble de permissions. Cette séparation est essentielle pour la sécurité et la stabilité du système.

Dans cette section, nous verrons :

- la différence entre **root** et les **utilisateurs standards** ;
- le rôle des fichiers **/etc/passwd** et **/etc/group** ;
- les principales **commandes de gestion des comptes**.

Comptes root et utilisateurs standards

Le compte root

- C'est le **super-utilisateur** du système.
- Il a **tous les droits**, sans restriction : il peut installer, modifier, ou supprimer n'importe quel fichier.
- Son répertoire personnel est généralement `/root`.
- On l'utilise uniquement pour des tâches d'administration.

Pour exécuter une commande en tant que root :

```
sudo commande
```

Exemple :

```
sudo apt update
```

Pour ouvrir une session root :

```
sudo -i
```

ou, sur certains systèmes :

```
su -
```

□□ **Attention :** une erreur avec le compte root peut rendre le système inutilisable. Toujours préférer `sudo` à une connexion directe en root.

Les utilisateurs standards

• Ce sont les comptes créés pour les personnes ou les services.

• Chaque utilisateur possède :

• un identifiant unique (**UID**) ;

• un groupe principal (**GID**) ;

• un répertoire personnel (souvent `/home/nom_utilisateur`) ;

• un shell par défaut (souvent `/bin/bash`).

L'utilisateur ne peut pas modifier les fichiers système sans droits administratifs.

Pour connaître le nom de l'utilisateur actuel :

```
whoami
```

Pour afficher des informations sur son compte :

```
id
```

Les fichiers de configuration

`/etc/passwd`

Ce fichier contient les informations de base sur tous les comptes utilisateurs.

Afficher son contenu :

```
cat /etc/passwd
```

Chaque ligne correspond à un utilisateur et contient plusieurs champs séparés par des : :

```
nom_utilisateur:x:UID:GID:commentaire:/home/nom:/bin/bash
```

Exemple :

```
kaya:x:1000:1000:Samuel Guillard:/home/kaya:/bin/bash
```

- **nom_utilisateur** : nom du compte
- **x** : mot de passe chiffré (stocké dans `/etc/shadow`)
- **UID** : identifiant de l'utilisateur
- **GID** : identifiant du groupe principal
- **commentaire** : informations facultatives
- **/home/nom** : répertoire personnel
- **/bin/bash** : shell par défaut

`/etc/group`

Ce fichier définit les groupes du système. Chaque ligne contient la liste des membres d'un groupe :

```
nom_du_groupe:x:GID:liste_utilisateurs
```

Exemple :

```
sudo:x:27:kaya
```

Cela signifie que l'utilisateur **kaya** appartient au groupe **sudo**.

Pour afficher les groupes d'un utilisateur :

```
groups kaya
```

Commandes de gestion des utilisateurs

`useradd` — créer un nouvel utilisateur

Crée un utilisateur avec un répertoire personnel et un shell par défaut.

```
sudo useradd -m -s /bin/bash alice
```

- `-m` : crée automatiquement le dossier `/home/alice`
- `-s` : spécifie le shell de connexion

`passwd` — définir ou changer un mot de passe

```
sudo passwd alice
```

Le système demande alors d'entrer et de confirmer le mot de passe.

Pour modifier son propre mot de passe :

```
passwd
```

`usermod` — modifier un utilisateur existant

Modifier le nom, le shell ou ajouter un utilisateur à un groupe :

```
sudo usermod -l nouveau_nom alice
sudo usermod -s /bin/zsh alice
sudo usermod -aG sudo alice
```

- `-l` : change le nom d'utilisateur
- `-s` : change le shell
- `-aG` : ajoute l'utilisateur à un groupe (sans retirer les autres)

`groupadd` — créer un nouveau groupe

Crée un groupe système ou un groupe d'utilisateurs personnalisé.

```
sudo groupadd compta
```

- `groupadd` : commande utilisée pour créer un groupe.
- `compta` : nom du groupe créé, qui sera ajouté au fichier `/etc/group`.

Pour vérifier la création du groupe :

```
getent group compta
```

Et pour supprimer un groupe :

```
sudo groupdel compta
```

`deluser` — supprimer un utilisateur

Supprime un compte utilisateur :

```
sudo userdel alice
```

Supprimer aussi son répertoire personnel :

```
sudo userdel -r alice
```

Consultation et vérification

Lister les utilisateurs actuellement connectés :

```
who
```

Afficher des informations détaillées sur les sessions actives :

```
loginctl list-sessions
```

Utiliser su et sudo

```
id
groups
sudo whoami
su -
```

`sudo` exécute une commande selon les autorisations du compte, en demandant habituellement son mot de passe. `su -` ouvre une session de connexion pour root et demande normalement le mot de passe de root. Quitter cette session avec `exit`.

Sur Debian, les membres du groupe `sudo` sont généralement autorisés par la configuration fournie. La politique se consulte dans `/etc/sudoers` et `/etc/sudoers.d/` ; utiliser `visudo` pour toute modification. `/etc/shadow` contient les empreintes de mots de passe et des informations d'expiration, et doit rester protégé.

À retenir

* Le compte **root** dispose de tous les droits : à utiliser avec précaution. * Les utilisateurs standards ont des droits limités et un répertoire personnel dans `/home`. * Les informations sur les utilisateurs et groupes sont stockées dans `/etc/passwd` et `/etc/group`. * Commandes principales :

- `useradd` : créer un utilisateur
- `passwd` : définir ou changer un mot de passe
- `usermod` : modifier un utilisateur existant

- `userdel` : supprimer un utilisateur
- Les groupes permettent de gérer les permissions collectivement.

20. Lire et modifier les permissions

Objectif Comprendre propriétaire, groupe et autres, puis attribuer des droits adaptés.

Repères LPI 101 : 104.5.

Commandes à connaître : `ls -l`, `chmod`, `chown`, `chgrp`.

Introduction

Sous Linux, **chaque fichier et répertoire appartient à un utilisateur et à un groupe**, et possède un ensemble de **droits d'accès**. Ces droits déterminent **qui peut lire, modifier ou exécuter** un fichier. Le système de permissions est l'un des fondements de la sécurité sous Linux.

Cette section aborde :

- les droits **r**, **w**, **x** sur les fichiers et répertoires ;
- les commandes `chmod`, `chown` et `chgrp` ;
- le masque `umask` *(optionnel)* ;
- les bits spéciaux `suid`, `sgid` et `sticky bit` *(optionnel)*.

Structure des permissions

Afficher les permissions d'un fichier ou dossier :

```
ls -l
```

Exemple de sortie :

```
-rwxr-xr--
```

Décomposons cette chaîne :

Position	Signification	Exemple
1er caractère	type d'objet : `-` (fichier), `d` (dossier), `l` (lien)	`-`
2-4	droits du **propriétaire**	`rwx`
5-7	droits du **groupe**	`r-x`

Position	Signification	Exemple
8-10	droits des **autres utilisateurs**	`r--`

Signification des droits

Droit	Valeur numérique	Signification sur un fichier	Signification sur un répertoire
`r`	4	lecture du contenu	lecture de la liste des fichiers
`w`	2	modification du contenu	création ou suppression de fichiers
`x`	1	exécution (programme ou script)	accès (possibilité de “rentrer” dans le dossier)

Exemples :

```
rwx------  → le propriétaire a tous les droits
rw-r--r--  → lecture/écriture pour le propriétaire, lecture seule pour les autres
rwxr-xr-x  → fichier exécutable pour tout le monde
```

LPI 101 / REPÈRE VISUEL

Lire les permissions sans mélanger les rôles

Exemple : -rwxr-xr--, soit 754 pour les neuf bits de permissions.

Type	Propriétaire	Groupe	Autres
-	rwx	r-x	r--
fichier	4 + 2 + 1	4 + 0 + 1	4 + 0 + 0

`r = 4 w = 2 x = 1 – = droit absent`

La même lettre change de sens selon l'objet.

Droit	Sur un fichier	Sur un répertoire
r	Lire son contenu	Lister les noms
w	Modifier son contenu	Créer / supprimer des entrées
x	Lancer le programme	Traverser le répertoire

Répertoire : w s'utilise généralement avec x pour créer ou supprimer.
Les permissions classiques choisissent une catégorie : propriétaire, groupe ou autres.

Les permissions du propriétaire, du groupe et des autres ; sens de r, w et x.

Commande `chmod` — modifier les permissions

`chmod` permet de modifier les droits d'un fichier ou d'un dossier.

Modes symbolique et numérique

1. Mode symbolique

```
chmod u+x script.sh
```

- `u` : utilisateur (owner)

- `g` : groupe
- `o` : autres
- `a` : tous
- `+` : ajouter un droit
- `-` : retirer un droit
- `=` : définir les droits exacts

Exemples :

```
chmod g+w fichier.txt     # ajoute l'écriture pour le groupe
chmod o-r fichier.txt     # retire la lecture aux autres
chmod a+x script.sh       # rend le fichier exécutable pour tous
```

2. Mode numérique

Chaque droit correspond à un chiffre :

- `r` = 4
- `w` = 2
- `x` = 1

Additionne les valeurs pour chaque catégorie :

Droits	Valeur
`rwx`	7
`rw-`	6
`r--`	4

Exemples :

```
chmod 600 fichier.txt   # lecture/écriture pour le propriétaire
chmod 755 script.sh     # rwx pour propriétaire, rx pour groupe et autres
chmod 644 document.txt  # rw- pour propriétaire, r-- pour les autres
```

Commande `chown` — changer le propriétaire

Changer le propriétaire d'un fichier :

```
sudo chown alice fichier.txt
```

Changer le propriétaire et le groupe :

```
sudo chown alice:developpeurs fichier.txt
```

Appliquer récursivement à un dossier et son contenu :

```
sudo chown -R alice:alice /home/alice
```

Commande `chgrp` — changer uniquement le groupe

```
sudo chgrp admin fichier.txt
```

Permet de modifier uniquement le groupe sans toucher au propriétaire.

Lire les droits d'un répertoire

Sur un **fichier**, `r` permet de lire le contenu, `w` de le modifier et `x` de tenter son exécution. Sur un **répertoire**, `r` permet de lister les noms, `w` de modifier ses entrées et `x` de le traverser et d'accéder aux éléments par leur nom.

La création et la suppression d'une entrée demandent généralement `w` et `x` sur le répertoire parent. Pouvoir supprimer un fichier ne dépend donc pas simplement du droit d'écriture sur le fichier lui-même. Le sticky bit ajoute une restriction, expliquée dans le chapitre suivant.

À retenir

`chmod` change les droits, `chown` le propriétaire et `chgrp` le groupe. Comprendre l'accès attendu avant de choisir une valeur numérique ; `777` est rarement une réponse adaptée.

21. Droits par défaut et bits spéciaux

Objectif Expliquer umask, SUID, SGID et sticky bit avec des cas concrets.

Repères LPI 101 : 104.5.

Commandes à connaître : `umask`, `chmod`.

Masque `umask`

`umask` retire des droits aux permissions demandées lors de la création d'un fichier ou d'un répertoire. Ce masque se combine par opérations sur les bits, pas par une soustraction arithmétique générale.

Afficher la valeur actuelle :

```
umask
```

Exemple :

```
0022
```

Interprétation :

- Fichier créé → permissions par défaut `644` avec un masque `022` (rw-r--r--)
- Répertoire créé → permissions par défaut `755` avec un masque `022` (rwxr-xr-x)

Modifier temporairement la valeur :

```
umask 002
```

Cela donnera par défaut :

- fichiers : `664`
- répertoires : `775`

Bits spéciaux : `suid`, `sgid`, `sticky bit`

Ces bits modifient le comportement d'exécution ou d'accès à certains fichiers.

`suid` (Set User ID)

- Sur un fichier exécutable : le programme s'exécute avec les droits du propriétaire.

• Exemple : `/usr/bin/passwd` possède le bit `s` pour root :

```
text -rwsr-xr-x 1 root root /usr/bin/passwd
```

Cela permet à un utilisateur de modifier son mot de passe sans être root.

`sgid` (Set Group ID)

• Sur un dossier : les nouveaux fichiers héritent du groupe du dossier.

• Exemple :

```
bash chmod g+s /srv/projets
```

Tous les fichiers créés dans `/srv/projets` appartiendront au même groupe.

`sticky bit`

• Dans un dossier partagé, réserve la suppression ou le renommage d'une entrée au propriétaire du fichier, au propriétaire du dossier ou à un processus suffisamment privilégié.

• Utilisé dans `/tmp` :

```
text drwxrwxrwt 10 root root /tmp
```

Le `t` final indique le sticky bit.

Calculer les droits à la création

Le masque retire des droits aux permissions demandées par le programme. Pour les créations courantes : base `666` pour un fichier ordinaire et `777` pour un dossier.

Masque	Fichier courant	Dossier courant
`022`	`644`	`755`
`027`	`640`	`750`
`077`	`600`	`700`

`umask` ne modifie pas les fichiers déjà existants et n'ajoute pas de droit d'exécution aux nouveaux fichiers ordinaires. Les ACL par défaut peuvent aussi intervenir.

À retenir

`umask` concerne les droits à la création ; SGID sur un dossier facilite le partage de groupe ; le sticky bit protège la suppression des entrées d'un dossier partagé. SUID sur un exécutable change l'identité effective du processus.

22. Observer et contrôler les processus

Objectif Identifier un processus, le suspendre, le reprendre et lui demander de s'arrêter.

Repères LPI 101 : 103.5.

Commandes à connaître : `ps`, `top`, `jobs`, `bg`, `fg`, `kill`, `pgrep`, `pkill`, `nohup`.

Introduction

Sous Linux, **un processus** correspond à un programme en cours d'exécution. Les commandes externes sont exécutées dans des processus ; certaines commandes internes, comme `cd`, agissent directement dans le shell courant. Le système permet d'en **afficher la liste**, de **contrôler leur exécution** et de **modifier leur priorité**.

Dans cette section, nous verrons :

- comment **consulter** les processus actifs ;
- comment **contrôler** leur exécution (arrêter, suspendre, relancer) ;
- comment **ajuster la priorité** d'un processus.

LPI 101 / REPÈRE VISUEL

Un shell, des processus et un terminal

Les PID ci-dessous sont des exemples : ils changent à chaque exécution.



Deux lancements possibles depuis le même shell.

PID	Identifiant du processus.
PPID	Identifiant du processus parent.

fg ramène une tâche au premier plan ; jobs affiche les tâches du shell.

En arrière-plan, la commande peut encore écrire dans le terminal.

Un shell parent et des processus enfants au premier plan ou en arrière-plan.

Consulter les processus

`ps` — afficher les processus en cours

La commande `ps` affiche les processus associés à la session courante.

Exemples :

```
ps
```

Affiche les processus lancés depuis le même terminal.

```
ps -ef
```

Affiche **tous les processus** du système avec plus d'informations :

- UID : utilisateur propriétaire du processus
- PID : identifiant du processus
- PPID : identifiant du processus parent
- CMD : commande lancée

Rechercher un processus particulier :

```
ps -ef | grep ssh
```

`top` — surveiller les processus en temps réel

`top` affiche la liste des processus triés par utilisation CPU ou mémoire.

```
top
```

Touches utiles :

- `q` → quitter
- `P` → trier par utilisation CPU
- `M` → trier par utilisation mémoire
- `k` → tuer un processus (demande le PID)

`htop` — vue améliorée

`htop` est une version plus lisible et interactive de `top`. Il affiche les processus sous forme de tableau coloré et permet la navigation au clavier.

Installer (si nécessaire) :

```
sudo apt install htop
```

Lancer :

```
htop
```

Contrôle des processus

Chaque processus a un **PID (Process ID)** unique. On peut suspendre, mettre en arrière-plan ou arrêter un processus.

Lancer un processus en arrière-plan

Ajouter `&` à la fin d'une commande :

```
gedit &
```

Le programme se lance en arrière-plan et rend la main au terminal.

Lister les tâches du terminal :

```
jobs
```

Ramener une tâche en avant-plan :

```
fg %1
```

Renvoyer une tâche en arrière-plan :

```
bg %1
```

`kill` — terminer un processus

La commande `kill` envoie un signal à un processus.

Exemples :

```
kill 1234
```

(envoie par défaut le signal `SIGTERM` pour demander l'arrêt proprement)

Forcer l'arrêt d'un processus bloqué :

```
kill -9 1234
```

(`-9` correspond au signal `SIGKILL` — arrêt immédiat sans nettoyage)

Envoyer un signal aux processus correspondant à un nom :

```
pkill firefox
```

Suspendre et reprendre une tâche

```
sleep 300
```

Pendant l'attente, **Ctrl+Z** suspend la tâche. Puis saisir :

```
jobs
bg %1
fg %1
```

Adapter `%1` au numéro affiché par `jobs`. `bg` reprend une tâche suspendue en arrière-plan ; `fg` la place au premier plan. Terminer le `sleep` au premier plan avec **Ctrl+C**. Un numéro de tâche du shell n'est pas un PID.

Laisser une commande continuer après la déconnexion

```
nohup sleep 300 > attente.log 2>&1 &
echo $!
```

`nohup` fait ignorer le signal de déconnexion SIGHUP et `&` rend la main au terminal. `$!` affiche le PID du dernier processus lancé en arrière-plan. Cela ne protège pas d'un arrêt de la machine ni d'une politique de session qui termine les processus.

Retrouver un processus

```
pgrep -a sleep
ps -eo pid,ppid,user,stat,ni,comm --sort=comm
```

`pgrep` recherche par nom ; `pkill` envoie un signal aux processus correspondant au nom. Vérifier la sélection avant d'agir. Les multiplexeurs `screen` et `tmux` permettent de détacher puis retrouver une session de terminal ; leur utilisation détaillée reste un prolongement.

À retenir

> Ctrl+C demande une interruption ; Ctrl+Z suspend ; `bg` reprend en arrière-plan ; `fg` ramène au premier plan. Préférer une demande d'arrêt normale à `kill -9`.

23. Comprendre les priorités des processus

Objectif Lire la valeur de niceness et diminuer la priorité d'une tâche coûteuse.

Repères LPI 101 : 103.6.

Commandes à connaître : `nice`, `renice`, `ps`.

Priorité des processus

Tous les processus ont une **priorité d'exécution**. Plus la priorité est élevée, plus le processus obtient du temps CPU. Sous Linux, cette priorité est définie par une valeur appelée **"niceness"**.

`nice` — lancer un processus avec une priorité donnée

Les valeurs possibles vont de **-20 (plus prioritaire)** à **19 (moins prioritaire)**. La valeur par défaut est **0**.

Lancer un processus avec une priorité basse :

```
nice -n 10 commande
```

Exemple :

```
nice -n 10 tar -czf sauvegarde.tar.gz /home
```

`renice` — modifier la priorité d'un processus existant

Changer la priorité d'un processus en cours :

```
sudo renice -n 5 -p 1234
```

Affiche la priorité actuelle et la modifie si autorisé. Augmenter la priorité demande des autorisations adaptées ; le cas courant est une commande lancée avec les privilèges administrateur.

Lire la valeur NI

```
nice -n 10 sleep 300 &
pid_tache=$!
ps -o pid,ni,comm -p "$pid_tache"
renice -n 15 -p "$pid_tache"
kill "$pid_tache"
```

Pour les tâches ordinaires, une valeur NI plus élevée correspond à une moindre priorité relative. L'effet apparaît surtout lorsque des tâches se disputent le CPU ; ce réglage ne fixe pas un pourcentage de processeur. Les enfants héritent normalement de la niceness du parent. `nice -n 10` ajoute 10 à la valeur héritée ; `renice -n 15` fixe ici la valeur à 15.

À retenir

Sans autorisation particulière, un utilisateur peut généralement rendre ses tâches moins prioritaires. Abaisser la valeur NI pour augmenter leur priorité demande des privilèges ou des limites adaptées.

24. Se repérer dans l’arborescence Linux

Objectif Associer les répertoires importants à leur rôle.

Repères LPI 101 : 104.7.

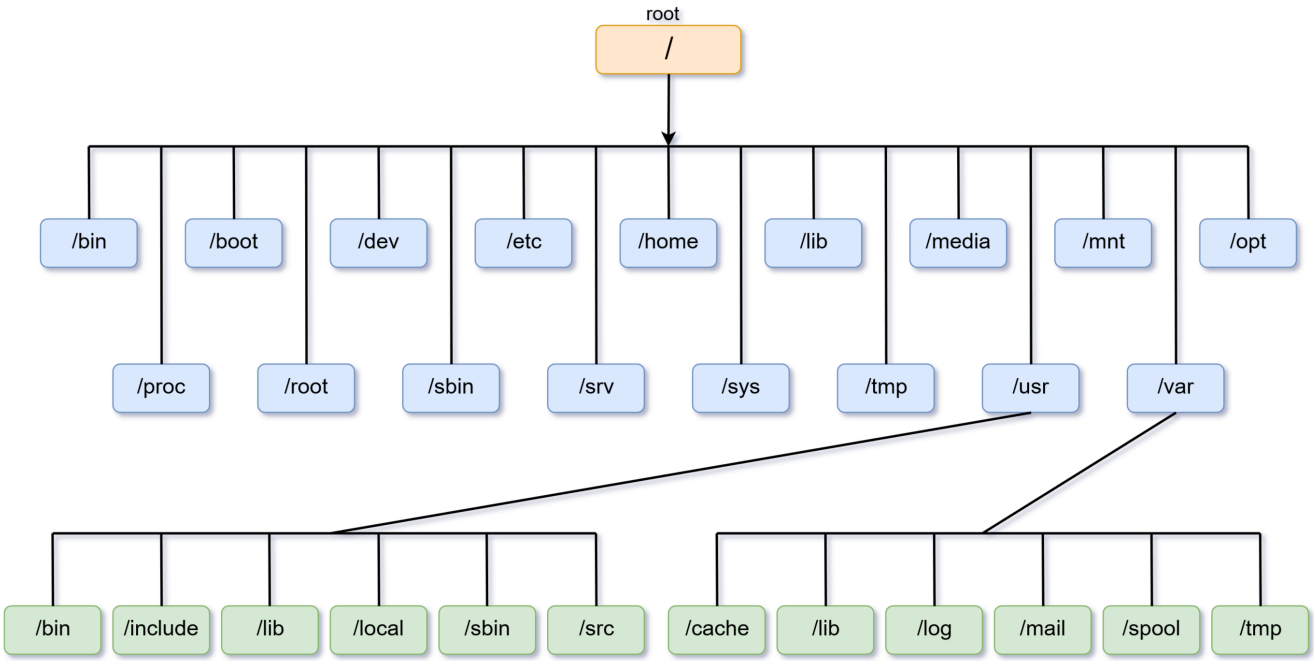
Introduction

Cette section s’inspire de l'exellent support de Hughes Bigo et présente la structure d’un système Linux selon le **FHS (Filesystem Hierarchy Standard)**. - https://hedgedoc.dawan.fr/wskoGbqwRcSkQBoC9RG_KA?both

Sous Linux, tout est fichier et l’arborescence unique part de la racine `/`. Le FHS décrit quels répertoires existent, ce qu’ils contiennent et à quoi ils servent. Pour la référence locale : `man hier`.

Structure FHS (Filesystem Hierarchy Standard)

Le FHS définit une arborescence standardisée afin de garantir la cohérence entre distributions.



Répertoires principaux et leur rôle

- **/** Racine de l'arborescence. Tous les chemins en dépendent.
- **/bin**, **/sbin**, **/lib**, **/lib64** Binaires et bibliothèques essentiels (souvent redirigés via usrmerge vers `/usr/bin`, `/usr/sbin`, `/usr/lib`, etc.). Utilisés très tôt au démarrage.
- **/boot** Fichiers nécessaires au démarrage (noyaux, initrd, chargeur).
- **/dev** Fichiers de périphériques (caractère/bloc), gérés dynamiquement par `udev` (ex. `/dev/sda`, `/dev/null`).
- **/etc** Configuration système spécifique à l'hôte. Fichiers statiques de configuration (pas de binaires).
- **/home** Répertoires personnels des utilisateurs finaux (`/home/<user>`).
- **/root** Répertoire personnel de l'administrateur `root` (séparé de `/home`).
- **/proc** Pseudo-FS fournissant des vues dynamiques du noyau et des processus (ex. `/proc/cpuinfo`, `/proc/meminfo`, `/proc/<PID>`).
- **/sys** Pseudo-FS exposant le modèle des périphériques et aspects du noyau (ex. `/sys/class/net`, `/sys/block`).
- **/run** Données d'exécution (runtime) depuis le dernier boot. Volatile.
- **/tmp** Fichiers temporaires (souvent `tmpfs`, nettoyage fréquent). Droits généralement `1777` (sticky bit).
- **/var** Données variables : logs, caches, spool, bases d'état, etc. Sous-répertoires usuels :
- `/var/log` : journaux

- `/var/lib` : données persistantes d'applications
- `/var/cache` : données régénérables
- `/var/tmp` : temporaires conservés plus longtemps que `/tmp`
- **/usr** Hiérarchie "secondaire" (majorité des programmes, bibliothèques, données partagées) :
- `/usr/bin`, `/usr/sbin` : binaires non essentiels
- `/usr/lib` : bibliothèques
- `/usr/share` : données indépendantes de l'architecture
- `/usr/local` : installations locales spécifiques à l'hôte
- **/opt** Logiciels additionnels "autocontenants" installés par paquet hors gestionnaire natif.
- **/srv** Données servies par des services (HTTP, FTP, VCS…).
- **/media** Points de montage pour supports amovibles (clés USB, CD/DVD), souvent créés automatiquement.
- **/mnt** Point(s) de montage temporaires utilisés par l'admin pour des montages manuels.
- **/lost+found** Présent à la racine de chaque système de fichiers ext*, destiné à la récupération après `fsck`.
- (**Spécifiques distribution**) **/snap** (Ubuntu Snap), éventuellement d'autres répertoires propres à l'écosystème (Flatpak, etc.).

Points clés modernes :

- **usrmerge** : sur de nombreuses distributions, `/bin`, `/sbin`, `/lib`, `/lib64` sont des liens symboliques vers leurs équivalents sous `/usr` (ex. `/usr/bin`), afin de simplifier la maintenance.
- **/run** : remplace `/var/run` pour les données d'exécution temporaires (vidées au démarrage).
- **/proc** et **/sys** : systèmes de fichiers virtuels exposant respectivement des informations sur les processus/le noyau et sur les périphériques/driver model.

Points de montage

Un **point de montage** est un répertoire auquel on rattache (monte) un système de fichiers. L'ensemble forme une seule arborescence à partir de `/`. Usages typiques :

- Monter des partitions dédiées : `/boot`, `/home`, `/var` (isolation, sécurité, quotas, redimensionnement plus simple).
- Monter des supports amovibles sous `/media/<label>`.
- Utiliser `/mnt` pour des montages ponctuels.

Commandes d’observation usuelles (aperçu, sans détailler le paramétrage qui sera vu en 5.2) :

- `lsblk` : visualiser disques/partitions et leurs points de montage.
- `findmnt` : lister les systèmes de fichiers montés et leur hiérarchie.
- `df -h` : voir l’espace utilisé/disponible par système de fichiers.

Sources externes - Page du manuel : `man hier` - wikipedia : https://fr.wikipedia.org/wiki/Filesystem_Hierarchy_Standard - Doc Ubuntu : https://doc.ubuntu-fr.org/fhs

---

À retenir

* Le **FHS** normalise l’arborescence pour assurer cohérence et portabilité. * Des répertoires sont **virtuels** (`/proc`, `/sys`), d’autres **volatils** (`/run`, souvent `/tmp`). * Les binaires essentiels et bibliothèques sont accessibles très tôt au boot ; de plus en plus de systèmes utilisent **usrmerge** pour centraliser sous `/usr`. * Les données variables et journaux résident sous `/var`, les profils utilisateurs sous `/home`, la configuration sous `/etc`. * Un **point de montage** attache un système de fichiers à un répertoire dans l’unique arborescence qui part de `/`. * Pour explorer : `lsblk`, `findmnt`, `df -h` donnent une vue rapide des disques et montages.

25. Concevoir un partitionnement simple

> Objectif Choisir les points de montage en fonction de l'usage de la machine.

Repères LPI 101 : 102.1.

Commandes à connaître : `lsblk -f`, `swapon --show`.

Comprendre le partitionnement

Lors de l'installation, il faut définir **comment le disque sera découpé**. Chaque partition aura un **point de montage**, c'est-à-dire un dossier où le système l'intégrera.

Point de montage	Rôle principal	Taille conseillée
`/`	Racine du système (fichiers du noyau, binaires, etc.)	20 à 30 Go
`/home`	Données personnelles des utilisateurs	le reste du disque
`/boot`	Fichiers nécessaires au démarrage	selon les noyaux conservés et la distribution
`swap`	Espace d'échange (pas un point de montage)	selon RAM, charge et hibernation
`/var` *(optionnel)*	Journaux, caches et données variables	selon les services et leur croissance
`/srv` *(optionnel)*	Données des services (web, ftp, etc.)	selon besoin

---

Un exemple pour une VM de formation

Pour un disque virtuel de 40 Gio, un exemple possible en UEFI est une partition système EFI d'environ 512 Mio, une racine `/` d'environ 24 Gio, un espace swap adapté à la VM et le reste pour `/home`. Ces valeurs illustrent un choix ; elles ne constituent pas une règle universelle.

```
Disque virtuel
├── partition EFI → /boot/efi (FAT, démarrage UEFI)
├── partition ext4 → / (système)
├── partition ext4 → /home (données personnelles)
└── espace swap (partition ou autre organisation adaptée)
```

La partition EFI et `/boot` ont des rôles différents : `/boot` contient les noyaux et images de démarrage ; l'ESP contient les exécutables EFI. Une installation BIOS n'utilise pas une ESP pour son démarrage BIOS. En BIOS avec GPT, GRUB peut demander une petite partition BIOS Boot.

Pourquoi séparer des points de montage ?

Point	Intérêt d'une séparation	Question à se poser
`/home`	Séparer les données des comptes du système	Quelle place pour les fichiers des utilisateurs ?
`/var`	Isoler les écritures des journaux, caches et services	Quels services et quelle croissance prévue ?
`/boot`	Répondre à certains besoins de démarrage	Combien de noyaux faut-il conserver ?

Séparer apporte de l'isolation mais impose de répartir l'espace. Pour une première VM, le partitionnement guidé reste adapté ; on étudie ensuite le résultat avec `lsblk -f`.

Dimensionner le swap

Le swap aide à gérer la pression mémoire ; il ne remplace pas la RAM. Sa taille dépend de la mémoire disponible, de la charge et d'un éventuel besoin d'hibernation. Le rapport « une à deux fois la RAM » n'est pas une règle générale.

À retenir

Un point de montage est un endroit dans l'arborescence. Le choix des partitions dépend de l'usage et du mode de démarrage, pas seulement de la taille du disque.

26. Créer des partitions et des systèmes de fichiers

Objectif Distinguer disque, partition et système de fichiers avant de formater.

Repères LPI 101 : 104.1.

Commandes à connaître : `lsblk`, `blkid`, `fdisk`, `mkfs`, `mkswap`.

Introduction

Sous Linux, le stockage repose sur la notion de **systèmes de fichiers montés dans une arborescence unique**. Chaque disque ou partition doit être **formaté** avec un système de fichiers (ext4, xfs, btrfs, etc.) puis **monté** à un emplacement (point de montage). Cette section explique les outils permettant de visualiser, créer, monter et gérer ces systèmes de fichiers.



Les trois niveaux du stockage : disque, partitions et système de fichiers.

Outils de gestion des disques

`lsblk` — visualiser la structure des disques et partitions

Affiche l'arborescence des périphériques de stockage et leurs points de montage.

```
lsblk
```

Exemple de sortie :

```
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0  100G  0 disk
├─sda1   8:1    0   96G  0 part /
```

```
└─sda2   8:2    0    4G  0 part [SWAP]
sdb      8:16   0   10G  0 disk
```

Options utiles :

- `-f` : affiche le type de FS, les labels et UUID
- `-p` : affiche les chemins complets (`/dev/sda1`, etc.)

`blkid` — identifier les partitions et leurs métadonnées

Permet d'afficher le type de système de fichiers, les labels et les UUID.

```
sudo blkid
```

Exemple :

```
/dev/sda1: UUID="a7b5d..." TYPE="ext4" PARTUUID="..."
/dev/sda2: UUID="e1a0a..." TYPE="swap"
```

Les **UUID** sont souvent utilisés dans `/etc/fstab` car ils restent stables même si le nom du périphérique change.

`fdisk` — gérer la table de partitions (MBR ou GPT)

Permet de visualiser ou modifier la table des partitions d'un disque.

Lister la table de partitions :

```
sudo fdisk -l
```

Lancer le mode interactif sur un disque :

```
sudo fdisk /dev/sdb
```

Principales commandes (à taper dans l'interface fdisk) :

- `n` → nouvelle partition
- `d` → supprimer une partition
- `p` → afficher la table
- `w` → enregistrer et quitter
- `q` → quitter sans enregistrer

□□ **Attention** : ces opérations modifient la structure du disque, à manipuler uniquement sur un disque de test ou non critique.

Création et formatage d’un système de fichiers

`mkfs` — créer un système de fichiers

Formate une partition pour y créer un système de fichiers.

Exemples :

```
sudo mkfs.ext4 /dev/sdb1
sudo mkfs.xfs /dev/sdb2
```

Autres formats possibles :

- `ext4` (par défaut sur Debian/Ubuntu)
- `xfs` (performant pour gros fichiers)
- `vfat` / `exfat` (compatibles Windows)
- `btrfs` (système moderne à snapshots)

`mkswap` — initialiser une partition de swap

Le swap est un espace disque utilisé comme mémoire d’appoint.

```
sudo mkswap /dev/sdb3
sudo swapon /dev/sdb3
sudo swapoff /dev/sdb3
```

Introduction à LVM et Btrfs

- **LVM (Logical Volume Manager)** : permet de gérer l’espace disque de manière flexible (agrandir, réduire, regrouper des disques sans repartitionner). Commandes : `pvcreate`, `vgcreate`, `lvcreate`, `lvextend`, `lvreduce`.
- **Btrfs** : système de fichiers moderne avec gestion intégrée des snapshots, sous-volumes, compression et vérification automatique.

Ces sujets sont abordés en détail dans des formations d’administration avancée.

Choisir une table de partitions et un système de fichiers

GPT est la table de partitions moderne ; **MBR** reste à reconnaître pour les systèmes plus anciens. La table décrit les partitions ; le système de fichiers organise les données à l'intérieur d'une partition ou d'un volume.

Système de fichiers	Repère
ext2 / ext3 / ext4	Famille Linux ; ext3 et ext4 proposent un journal.
XFS	Système journalisé fréquent sur les serveurs ; outils de maintenance propres.
FAT / VFAT	Compatibilité et partition système EFI.
exFAT	Échanges de fichiers, notamment sur supports amovibles.
Btrfs	Sous-volumes, compression et fonctions sur plusieurs périphériques.

Les commandes correspondantes comprennent `mkfs.ext4`, `mkfs.xfs`, `mkfs.vfat` et `mkfs.exfat`, selon les outils installés. Pour débuter, se concentrer sur ext4.

À retenir

Les exemples de création et formatage s'appliquent à un **disque supplémentaire de laboratoire vide**, identifié avant la commande. Le nom `/dev/sdb` est un exemple ; il peut être différent dans votre VM.

27. Monter un système de fichiers et utiliser fstab

> Objectif Rendre un système de fichiers accessible puis comprendre son montage au démarrage.

Repères LPI 101 : 104.3.

Commandes à connaître : `mount`, `umount`, `findmnt`, `blkid`.

Fichiers et emplacements : `/etc/fstab`.

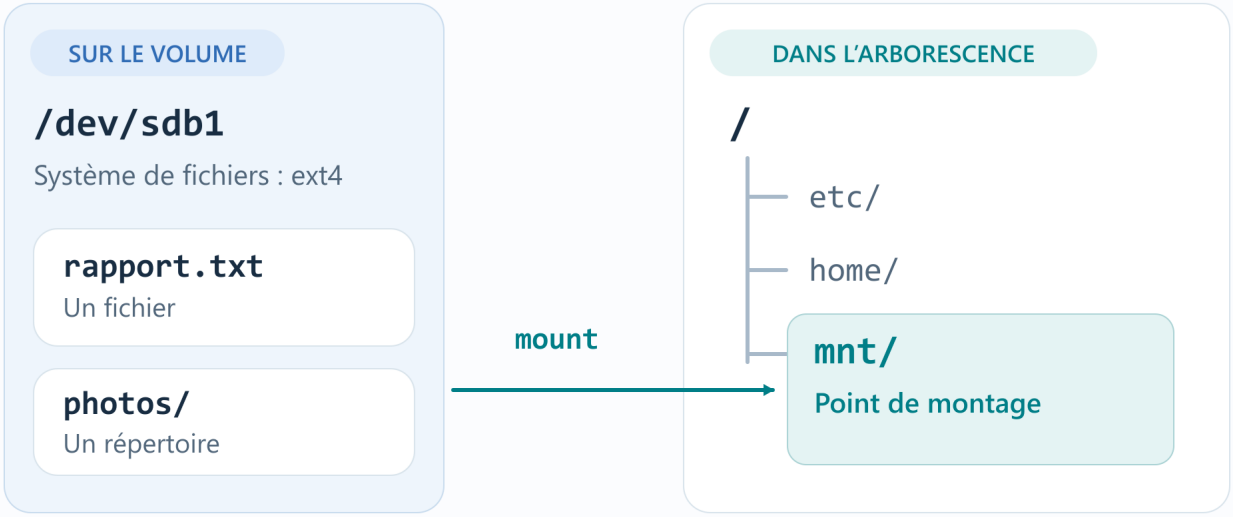
Montage et démontage

Un **système de fichiers** doit être monté dans un répertoire pour être accessible.

LPI 101 / REPÈRE VISUEL

Monter : donner un chemin aux fichiers

Le volume garde ses données ; un répertoire devient leur point d'accès.




```
sudo mount /dev/sdb1 /mnt
```

Après le montage, on accède aux fichiers par ces chemins :

`/mnt/rapport.txt` `/mnt/photos/`

Monter ne copie pas les fichiers.
Démonter enlève cet accès ; les données restent sur le volume.

Un volume ext4 sur /dev/sdb1 devient accessible sous /mnt après son montage. Les fichiers restent sur le volume.

Le répertoire `/mnt` sert ici de porte d'entrée vers les fichiers du volume : monter ne déplace pas les données.

`mount` — monter un système de fichiers

```
sudo mount /dev/sdb1 /mnt
```

→ `/mnt` devient le point d'accès à la partition.

Afficher les systèmes montés :

```
mount | grep "^/dev"
```

`umount` — démonter un système de fichiers

```
sudo umount /mnt
```

□□ Ne pas oublier de fermer les fichiers ouverts avant de démonter. Si un message indique que le périphérique est occupé :

```
sudo lsof +f -- /mnt
```

Montage automatique avec `/etc/fstab`

Le fichier `/etc/fstab` définit les systèmes de fichiers montés automatiquement au démarrage.

Exemple d'entrée :

```
UUID=a7b5d-1234   /data   ext4   defaults   0   2
```

Colonnes :

1. périphérique ou UUID
2. point de montage
3. type de FS
4. options (`defaults`, `ro`, `noexec`, `user`, etc.)
5. dump (0 = non)
6. ordre de vérification (0 = désactivé, 1 = racine, 2 = autres)

Tester une ligne avant redémarrage :

```
sudo mount -a
```

Choisir les options et identifier le volume

`ro` demande un montage en lecture seule ; `rw`, en lecture-écriture. `noexec`, `nosuid` et `nodev` limitent respectivement l'exécution directe, les bits SUID/SGID et l'interprétation des fichiers de périphériques sur le montage.

```
lsblk -f
findmnt /
```

Un UUID identifie un système de fichiers de manière plus stable qu'un nom tel que `/dev/sdb1`. Une étiquette peut être utilisée sous la forme `LABEL=donnees`. Les six champs de `fstab` décrivent la source, le point de montage, le type, les options, le champ dump et l'ordre de vérification.

Vérifier avant de redémarrer

Après modification de `/etc/fstab`, garder une console ouverte, vérifier les chemins et tester le montage voulu. `mount -a` tente les montages automatiques de `fstab` mais ne simule pas complètement un redémarrage et ne remonte pas forcément ce qui est déjà monté.

```
sudo systemctl daemon-reload
sudo mount -a
findmnt
```

Sous systemd, les entrées de `fstab` sont converties en unités `.mount`. Des unités `.automount` peuvent déclencher un montage à l'accès ; leur écriture manuelle reste un approfondissement.

À retenir

Formater prépare le contenu ; monter le rend accessible à un point de l'arborescence. Démonter avant de retirer un support ; en cas de volume occupé, quitter les dossiers et applications qui l'utilisent.

28. Mesurer l’espace, la mémoire et les inodes

Objectif Distinguer capacité du système de fichiers, taille d’un dossier et mémoire disponible.

Repères LPI 101 : 104.2.

Commandes à connaître : `df`, `du`, `free`, `swapon`.

Gestion de l’espace disque

`df` — affichage de l’espace disque par système de fichiers

```
df -h
```

Affiche la taille totale, utilisée et libre sur chaque système de fichiers.

Exemple :

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        96G   12G   79G  14% /
tmpfs           1.9G  2.0M  1.9G   1% /run
```

`du` — taille d’un répertoire ou d’un fichier

```
du -sh /var/log
```

- `-s` : total uniquement
- `-h` : taille lisible (Ko, Mo, Go)

Lister les plus gros dossiers :

```
du -h --max-depth=1 /home | sort -hr | head
```

`free` — état de la mémoire et du swap

```
free -h
```

Affiche la mémoire utilisée, libre et partagée, ainsi que la taille du swap.

Exemple :

```
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       1.2Gi       1.5Gi       110Mi       1.1Gi       2.3Gi
Swap:          4.0Gi          0B       4.0Gi
```

Suivi de l’espace et des inodes

Chaque fichier du système possède un **inode** — une entrée contenant ses métadonnées. Même avec de l’espace disque libre, un système peut être bloqué si tous les inodes sont utilisés.

Vérifier leur usage :

```
df -i
```

Exemple :

```
Filesystem     Inodes  IUsed   IFree IUse% Mounted on
/dev/sda1      655360  49217  606143    8% /
```

> Si **IUse%** atteint 100%, cela signifie que trop de petits fichiers ont été créés (souvent dans `/var/log` ou `/tmp`).

Choisir le bon indicateur

Question	Commande
Le système de fichiers est-il plein ?	`df -h`
Quel dossier occupe de la place ?	`du -sh dossier`
Reste-t-il des inodes ?	`df -i`
Quelle mémoire est disponible ?	`free -h`
Quel swap est actif ?	`swapon --show`

`df` et `du` mesurent des choses différentes. Un fichier supprimé mais encore ouvert peut continuer à occuper de l’espace ; les métadonnées et blocs réservés expliquent aussi certaines différences.

À retenir

Un système peut manquer d’inodes alors qu’il reste des octets libres. Ne pas confondre espace disque et mémoire vive.

29. Vérifier et entretenir les systèmes de fichiers

Objectif Comprendre les outils d'inspection et les conditions d'une vérification.

Repères LPI 101 : 104.2.

Commandes à connaître : `tune2fs`, `fsck`, `e2fsck`, `rsync`.

Introduction

Même si les systèmes de fichiers Linux comme **ext4** ou **xfs** sont très fiables, il peut arriver qu'un volume soit endommagé après une coupure brutale ou un problème matériel. Cette partie explique comment **observer** l'état d'un système de fichiers et, de manière encadrée, comment **lancer une vérification** en cas de besoin.

□□ Avertissement important

Les outils de vérification (`fsck`, `e2fsck`) peuvent **modifier la structure interne du système de fichiers**. □ **Ne les utilisez jamais sur une partition montée** ou sur le disque système actif. Ils doivent être réservés :

- aux partitions de test, ou
- à une utilisation en **mode maintenance (live CD)**.

Dans le cadre de cette formation, **on se limite à la consultation et à la compréhension** de leur usage.

Vérification et inspection d'un système de fichiers

`tune2fs` — afficher les informations d'un volume ext4

Avec l'option `-l`, `tune2fs` lit les **métadonnées de maintenance** d'un système de fichiers ext2/3/4 sans modifier ses paramètres.

```
sudo tune2fs -l /dev/sdb1
```

Exemples de champs utiles :

- **Filesystem state** : indique si le système est propre (`clean`) ou s'il a besoin d'une vérification.
- **Last checked** : date de la dernière vérification.
- **Mount count / Maximum mount count** : nombre de montages avant vérification automatique.

`tune2fs -l` affiche des informations. D'autres options de `tune2fs` modifient les paramètres du système de fichiers.

`fsck` — vérifier la cohérence d'un système de fichiers

`fsck` est un frontal qui appelle l'outil de vérification adapté au type de système de fichiers. Les options et possibilités dépendent de cet outil ; les exemples suivants concernent ext4, démonté, dans le laboratoire.

```
sudo fsck -n /dev/sdb1
```

Options courantes :

- `-n` : demande une vérification sans réparation, selon l'outil appelé ; conserver le volume démonté
- `-y` : répond "oui" automatiquement aux corrections
- Vérifier les options du vérificateur concerné dans son manuel ; ne pas automatiser les réparations pendant cette initiation.

□□ N'utilisez **jamais** `fsck` **sur une partition montée**. Pour une vérification en sécurité : démontez d'abord le volume :

```
sudo umount /dev/sdb1
```

`e2fsck` — version spécifique pour ext2/ext3/ext4

`e2fsck` est la version dédiée des systèmes de fichiers ext*. `fsck` appelle normalement `e2fsck` pour un système ext2/3/4. L'utiliser directement permet de choisir ses options propres.

Exemple :

```
sudo e2fsck -fn /dev/sdb1
```

Options :

- `-f` : force la vérification même si le système est marqué "clean".
- `-n` : lecture seule, sans réparation.

Sauvegarde et prévention

Avant toute manipulation ou vérification, il est essentiel de **sauvegarder les données importantes**. Deux commandes simples permettent de le faire :

Copie complète d'un répertoire

```
cp -rpv /home/sam/Documents /mnt/backup/
```

- `-r` : récursif
- `-p` : conserve les droits et dates
- `-v` : affiche les fichiers copiés

Synchronisation rapide avec `rsync`

```
rsync -av /home/sam/Documents /mnt/backup/
```

- `-a` : mode archive
- `-v` : mode verbeux
- `--delete` (optionnel) : synchronise en supprimant les fichiers disparus côté source

---

À retenir

* `tune2fs -l` : affiche l'état d'un système de fichiers ext4 sans le modifier. * `fsck` / `e2fsck` : vérifient et réparent la cohérence d'un volume (uniquement quand il est démonté). * `df -i` : indique l'utilisation des inodes (utile pour diagnostiquer un disque "plein" de petits fichiers). * Toujours **sauvegarder avant de corriger** : `cp -r` ou `rsync` permettent de copier les données ; contrôler la destination et vérifier la restauration. * En cas de doute, **ne jamais exécuter une vérification sur le disque système actif** — passer par un live CD ou un mode de secours.

30. Installer et gérer les paquets Debian

Objectif Utiliser APT et distinguer dépôts, index, dépendances et paquets locaux.

Repères LPI 101 : 102.4.

Commandes à connaître : `apt`, `apt-get`, `apt-cache`, `dpkg`, `apt-mark`.

Introduction

Sous Debian/Ubuntu, l'écosystème **APT** gère les dépôts, la résolution des dépendances et les mises à jour. La couche basse, **dpkg**, manipule les paquets `.deb` localement (installation, liste des fichiers, état…).

Vocabulaire rapide

- **Paquet** : archive `.deb` contenant fichiers + scripts d'installation.
- **Dépôt** : source de paquets décrite dans `/etc/apt/sources.list` ou dans des fichiers `.list` / `.sources` sous `/etc/apt/sources.list.d/`.
- **Index** : catalogue local des paquets récupéré par `apt update`.

Fichiers et emplacements clés

- **Sources** :
- `/etc/apt/sources.list` (format traditionnel, parfois absent)
- `/etc/apt/sources.list.d/*.list` et `*.sources` (dont les sources principales selon l'installation)
- **Clés de dépôt** : `/usr/share/keyrings/` pour les clés gérées par paquets, `/etc/apt/keyrings/` pour celles ajoutées par l'administrateur ; associer la clé au dépôt avec `Signed-By`
- **Cache APT** : `/var/lib/apt/lists/` (index), `/var/cache/apt/archives/` (paquets téléchargés)
- **État dpkg** : `/var/lib/dpkg/` (`status`, listes des fichiers installés)

Cycle de base (APT)

1. **Mettre à jour l'index**

```
sudo apt update
```

1. **Mettre à niveau les paquets**

```
sudo apt upgrade              # met à niveau ; lire les changements proposés
sudo apt full-upgrade         # peut changer/désinstaller pour résoudre les deps
```

1. **Installer / supprimer**

```
sudo apt install htop curl
sudo apt remove htop          # supprime le paquet, garde les configs
sudo apt purge htop           # supprime aussi les fichiers de config
```

1. **Nettoyer**

```
sudo apt autoremove           # enlève les dépendances devenues inutiles
sudo apt clean                # vide les .deb téléchargés
```

Recherche et informations

```
apt search nginx              # chercher par nom/description
apt show nginx                # fiche détaillée (version, dépendances, mainteneur…)
apt list --installed          # lister les paquets installés
apt-cache policy nginx              # versions dispo, priorité (pin)
```

Astuce : pour lister rapidement les binaires installés par un paquet :

```
dpkg -L nginx | less          # liste les fichiers fournis par le paquet
```

Geler / tenir une version

Empêcher les mises à jour d'un paquet :

```
sudo apt-mark hold nginx
sudo apt-mark unhold nginx
```

Voir la liste des paquets geléss :

```
apt-mark showhold
```

Ajouter un dépôt tiers (méthode moderne)

Un dépôt tiers doit correspondre à la distribution et à sa version. Suivre les instructions de son éditeur ; conserver sa clé dans un fichier dédié et l'associer avec `Signed-By`. Éviter de rendre cette clé globalement fiable pour tous les dépôts.

Pour le parcours débutant, utiliser d'abord les dépôts fournis par Debian. Les PPA concernent Ubuntu et ne doivent pas être ajoutés tels quels à Debian.

Lire un fichier de dépôt .sources

Les nouveaux formats `.sources` remplacent progressivement les anciens fichiers `.list`. Ils regroupent les paramètres du dépôt en champs lisibles : URI, suites, composants et référence de clé ; ils peuvent aussi contenir une clé intégrée.

Exemple :

```
Types: deb
URIs: https://repo.example.com/debian
Suites: stable
Components: main
Signed-By: /usr/share/keyrings/repo.gpg
```

Sur les versions d'APT qui la proposent, la commande suivante convertit les sources existantes ; lire les modifications avant de l'utiliser :

```
sudo apt modernize-sources
```

Gestion directe de `.deb` (dpkg)

- **Installer un .deb local** :

```
sudo apt install ./package.deb   # recommandé (résout les dépendances)
# ou, couche basse :
sudo dpkg -i package.deb         # peut laisser des dépendances non satisfaites
sudo apt -f install              # corrige les manques après dpkg -i
```

- **Interroger dpkg** :

```
dpkg -l | less                   # liste des paquets (état)
dpkg -s nginx                    # état + méta du paquet installé
dpkg -L nginx                    # fichiers installés par nginx
dpkg -S /usr/sbin/nginx          # quel paquet possède ce fichier ?
```

Bonnes pratiques (prod & postes)

- Actualiser l'index avec `apt update` avant une session d'installation ou de mise à niveau.
- Préférer `apt install ./fichier.deb` plutôt que `dpkg -i`.
- Lire les installations et suppressions proposées par `apt full-upgrade`. Une montée de version de Debian suit les notes de publication de la version cible.

- Nettoyer régulièrement : `apt autoremove` et `apt clean`.
- Limiter le nombre de dépôts tiers, documenter leur ajout.

Retrouver le paquet fournissant un fichier

`dpkg -S /chemin/fichier` interroge les paquets installés. Pour chercher aussi dans les dépôts :

```
sudo apt install apt-file
sudo apt-file update
apt-file search /usr/bin/htop
```

`apt-get` et `apt-cache` sont également à reconnaître pour l'examen. `apt` convient aux usages interactifs du cours ; les scripts privilégient généralement les interfaces prévues pour eux, comme `apt-get`.

Documentation : sources.list — Debian.

À retenir

* **APT** gère les dépôts, les dépendances et les mises à jour, tandis que **dpkg** manipule directement les paquets `.deb`. * Cycle courant : `apt update` → `apt upgrade` / `apt full-upgrade` → `apt install / remove / purge` → `apt autoremove` / `apt clean`. * `apt show`, `apt search`, `apt policy` permettent d'obtenir des informations détaillées sur les paquets. * `apt-mark hold` permet de **geler** un paquet pour éviter sa mise à jour. * Les dépôts sont configurés dans `/etc/apt/sources.list` et `/etc/apt/sources.list.d/`. * Installer un `.deb` local avec `apt install ./fichier.deb` permet de résoudre automatiquement les dépendances.

31. Reconnaître RPM, DNF, YUM et Zypper

Objectif Retrouver les actions de gestion des paquets sur les autres grandes familles Linux.

Repères LPI 101 : 102.5.

Commandes à connaître : `rpm`, `dnf`, `yum`, `zypper`.

Introduction

Les distributions **de la famille Red Hat** (RHEL, AlmaLinux, Rocky Linux, Fedora) ainsi que **SUSE/openSUSE** n'utilisent pas les paquets `.deb`, mais des paquets **.rpm** (*Red Hat Package Manager*). Le principe reste le même : chaque paquet contient les fichiers d'un logiciel et les métadonnées nécessaires à son installation, sa mise à jour et sa suppression.

Gestionnaires de paquets

Niveau	Outil principal	Rôle
Bas niveau	`rpm`	installe, supprime, interroge directement les fichiers `.rpm` (contrôle les dépendances, sans les télécharger ni les résoudre automatiquement)
Haut niveau	`yum`, `dnf`, `zypper`	gèrent les dépôts, les dépendances, les mises à jour et la recherche

---

Le gestionnaire RPM (bas niveau)

• **Installer un paquet local**

```
bash sudo rpm -ivh paquet.rpm
```

(`-i` : installer, `-v` : verbeux, `-h` : barre de progression)

• **Mettre à jour / supprimer**

```
bash sudo rpm -Uvh paquet.rpm sudo rpm -e nom_du_paquet
```

• **Lister / interroger**

```
bash rpm -qa # tous les paquets installés rpm -qi nginx # infos détaillées rpm -ql nginx
# fichiers installés rpm -qf /usr/sbin/nginx # quel paquet possède ce fichier ?
```

□□ `rpm` ne résout **pas** les dépendances ; il faut donc préférer `dnf` ou `yum`.

Le gestionnaire DNF (ou YUM)

`dnf` (*Dandified YUM*) est l'évolution de `yum`. Il gère automatiquement les dépendances et succède à `yum` sur de nombreux systèmes.

- **Mettre à jour les dépôts et le système**

```
bash sudo dnf check-update sudo dnf upgrade
```

- **Installer / supprimer un paquet**

```
bash sudo dnf install htop sudo dnf remove htop
```

- **Rechercher un paquet**

```
bash dnf search nginx dnf info nginx
```

- **Nettoyer le cache**

```
bash sudo dnf clean all
```

- **Lister / verrouiller un paquet**

```
bash dnf list installed nginx sudo dnf versionlock add nginx # nécessite le module
versionlock adapté à la version de DNF
```

Sur **RHEL / AlmaLinux / Rocky**, `dnf` est standard. Sur **Fedora**, la version de DNF et les commandes de ses modules peuvent varier selon la version du système.

Le gestionnaire Zypper (openSUSE)

`zypper` est l'équivalent de `dnf` pour les distributions **SUSE**.

Exemples :

```
sudo zypper refresh             # mise à jour de l'index
sudo zypper install htop        # installation
sudo zypper remove htop         # suppression
sudo zypper search nginx        # recherche
sudo zypper update              # mise à jour système
```

Comparaison APT vs DNF/Zypper

Fonction	Debian / Ubuntu (APT)	Red Hat / SUSE (DNF / Zypper)
Fichier de sources	`/etc/apt/sources.list`	`/etc/yum.repos.d/*.repo` / `/etc/zypp/repos.d/`
Paquets	`.deb`	`.rpm`
Gestionnaire bas niveau	`dpkg`	`rpm`
Gestionnaire haut niveau	`apt` / `apt-get`	`dnf`, `yum`, `zypper`
Résolution dépendances	automatique (`apt`)	automatique (`dnf`/`zypper`)
Geler un paquet	`apt-mark hold`	`dnf versionlock add`

À retenir

- Les systèmes **Red Hat** utilisent des paquets **.rpm** au lieu de **.deb**.
- `rpm` agit au **niveau bas**, sans résolution automatique des dépendances.
- `dnf` (ou `yum`) est le **gestionnaire haut niveau** moderne ; `zypper` l'équivalent chez openSUSE.
- Les fichiers de configuration des dépôts se trouvent dans `/etc/yum.repos.d/` ou `/etc/zypp/repos.d/`.
- Les commandes sont globalement similaires à APT : `install`, `remove`, `search`, `update`, `clean`.
- Toujours préférer **dnf/yum/zypper** à `rpm` pour éviter les erreurs de dépendances.

32. Comprendre les bibliothèques partagées

Objectif Identifier une dépendance partagée et reconnaître les outils de chargement.

Repères LPI 101 : 102.3.

Commandes à connaître : `ldd`, `ldconfig`.

Fichiers et emplacements : `/etc/ld.so.conf`, `/etc/ld.so.conf.d/`, `/etc/ld.so.cache`.

Introduction

Sous Linux, les programmes utilisent souvent des **bibliothèques partagées** (fichiers `.so`) contenant du code commun (fonctions, outils…). Cela permet de **réduire la taille des exécutables**, **partager la mémoire** entre plusieurs programmes et **mettre à jour le code partagé sans recompiler**.

Deux types principaux existent :

- **Bibliothèques statiques (.a)** → incluses directement dans le binaire au moment de la compilation.
- **Bibliothèques dynamiques / partagées (.so)** → chargées par le système au moment de l'exécution.

Emplacements habituels

Répertoire	Rôle
`/lib`, `/lib64`	Bibliothèques essentielles au démarrage du système
`/usr/lib`, `/usr/lib64`	Bibliothèques pour les programmes standards
`/usr/local/lib`	Bibliothèques installées manuellement
`/opt/.../lib`	Bibliothèques spécifiques à un logiciel tiers

Identifier les bibliothèques d'un programme

Pour connaître les bibliothèques utilisées par un exécutable :

```
ldd /usr/bin/ls
```

Exemple de sortie :

```
linux-vdso.so.1 =>  (0x00007fffb7ffd000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2f46a00000)
```

Chaque ligne indique le nom de la bibliothèque et son chemin réel sur le disque.

Recherche et chargement des bibliothèques

Le **chargeur dynamique** (`ld.so` / `ld-linux.so`) trouve les bibliothèques nécessaires au lancement. Parmi les éléments qui interviennent : chemins enregistrés dans l'exécutable (RPATH/RUNPATH), variable `LD_LIBRARY_PATH`, cache `/etc/ld.so.cache` et répertoires standards.

`/etc/ld.so.conf` et `/etc/ld.so.conf.d/` sont lus par `ldconfig` pour construire le cache ; ce ne sont pas des fichiers relus de la même manière pour chaque lancement. L'ordre précis dépend notamment de RPATH/RUNPATH et du mode d'exécution sécurisé.

```
LD_LIBRARY_PATH=/opt/monapp/lib commande
ldconfig -p
```

La première forme ajuste l'environnement d'une seule commande. Elle illustre le mécanisme et suppose qu'un logiciel de ce nom est installé. Une bibliothèque manquante se résout d'abord en recherchant le paquet approprié.

Documentation : manuel du chargeur dynamique.

Mettre à jour le cache des bibliothèques

Les gestionnaires de paquets exécutent normalement les opérations nécessaires. Après une installation manuelle dans les chemins configurés, l'administrateur peut actualiser le cache :

```
sudo ldconfig
```

Cette commande met à jour le fichier `/etc/ld.so.cache`, qui contient la liste des chemins valides pour les bibliothèques partagées.

Pour vérifier que tout est bien pris en compte :

```
sudo ldconfig -v | grep lib
```

A retenir

- Les bibliothèques **statiques (.a)** sont intégrées dans le binaire.

- Les bibliothèques **dynamiques (.so)** sont chargées à l'exécution.
- `ldd` permet de lister les bibliothèques utilisées par un programme.
- `ldconfig` met à jour le cache du système.
- `LD_LIBRARY_PATH` permet d'ajouter temporairement un chemin personnalisé.

Utilisation de ldd Utiliser `ldd` sur les programmes connus de la VM, comme `/usr/bin/ls`, plutôt que sur un exécutable inconnu.

33. Identifier le matériel et les modules du noyau

> Objectif Observer les ressources de la VM et le rôle des pilotes.

Repères LPI 101 : 101.1.

Commandes à connaître : `lscpu`, `free`, `lspci`, `lsusb`, `lsmod`, `modinfo`, `modprobe`.

Introduction

Sous Linux, le matériel (processeur, mémoire, disques, périphériques USB, cartes réseau…) est entièrement accessible depuis le **système de fichiers virtuel** `/proc` et via des **outils d'inspection**. Comprendre ces commandes est essentiel pour diagnostiquer, surveiller et configurer un système.

Ce chapitre aborde :

- la **détection du matériel** ;
- la **vérification du processeur et de la mémoire** ;
- la **gestion des périphériques de stockage et USB** ;
- la **lecture des informations système via** `/proc` **et** `/sys`.

Informations sur le processeur

`lscpu` — afficher les informations CPU

```
lscpu
```

Affiche :

- modèle du processeur ;
- nombre de cœurs et de threads ;
- architecture (64 bits, ARM, etc.) ;
- hyper-threading, virtualisation, fréquences.

Exemple :

```
Architecture :            x86_64
Processeur(s) :           4
```

```
Modèle :                 Intel(R) Core(TM) i7-8650U CPU @ 1.90 GHz
Thread(s) par cœur :     2
Cœur(s) par socket :     2
```

Informations sur la mémoire

`free` — afficher la mémoire utilisée et disponible

```
free -h
```

- `-h` : affiche les valeurs dans un format lisible (Mo/Go).

`cat /proc/meminfo` — consulter le détail de la mémoire

```
cat /proc/meminfo | head
```

Donne des informations précises sur :

- la RAM totale et libre ;
- la mémoire tampon/cache ;
- la mémoire utilisée par le noyau.

Détection des périphériques matériels

Les périphériques PCI et USB peuvent être listés avec les commandes suivantes. Pour les disques, voir le chapitre 26 ; pour l'espace disque, le chapitre 28.

`lspci` — lister les périphériques PCI

```
lspci
```

Affiche les cartes réseau, contrôleurs SATA, GPU, etc. Pour plus de détails :

```
lspci -v
```

`lsusb` — lister les périphériques USB

```
lsusb
```

Affiche les clés USB, webcams, imprimantes, etc. Pour un affichage détaillé :

```
lsusb -v | less
```

`lsblk` — lister les périphériques de stockage (disques)

`lsblk` affiche les périphériques de stockage. Sa lecture détaillée, les partitions et les systèmes de fichiers sont traités dans le chapitre 26.

`df` — afficher l'espace disque utilisé

Voir le chapitre 28 pour mesurer l'espace disponible avec `df -h`.

`du` — mesurer l'espace occupé par un dossier

Voir le chapitre 28 pour mesurer la place occupée par un dossier avec `du -sh`.

Gestion des modules du noyau

Les **pilotes matériels** sous Linux sont chargés sous forme de **modules**.

`lsmod` — lister les modules chargés

```
lsmod
```

Affiche les modules du noyau actuellement en mémoire (pilotes actifs).

`modinfo` — obtenir des détails sur un module

```
sudo modinfo e1000e
```

Affiche la version, le fournisseur et les dépendances du module `e1000e` (carte Intel).

Arborescences virtuelles du système

`/proc`

Ce dossier contient des **fichiers virtuels** reflétant l'état du système (CPU, mémoire, processus…).

```
ls /proc
cat /proc/cpuinfo
cat /proc/meminfo
```

`/sys`

Ce dossier reflète la **hiérarchie des périphériques** détectés par le noyau (sysfs).

```
ls /sys/class/
ls /sys/block/
```

Comprendre le chargement d'un module

```
lsmod
modinfo loop
```

`lsmod` liste les modules chargés et `modinfo` décrit un module disponible. Un pilote intégré directement dans le noyau n'apparaît pas dans `lsmod`.

`sudo modprobe nom_module` charge un module avec ses dépendances ; `sudo modprobe -r nom_module` tente de le retirer s'il n'est pas utilisé. Ces formes servent de repères : conserver les pilotes du disque et du réseau actifs pendant la formation.

Périphériques et événements

`/dev` expose les nœuds de périphériques ; `/sys` décrit les objets du noyau ; `/proc` fournit des informations sur le système et les processus. **udev** gère les événements et noms de périphériques ; **D-Bus** est un mécanisme d'échange de messages entre composants de l'espace utilisateur.

Dans le firmware d'une machine physique, certains périphériques intégrés peuvent être activés ou désactivés. Dans une VM, les réglages du matériel virtuel sont effectués depuis l'hyperviseur. Reconnaître aussi les stockages SATA, SCSI, NVMe et USB ; leur nom dans `/dev` dépend du pilote et du type de matériel.

À retenir

- Linux expose toutes les informations matérielles via **/proc** et **/sys**. - Le CPU et la mémoire peuvent être consultés facilement depuis le système. - Les périphériques PCI, USB et disques sont détectés automatiquement par le noyau. - Les pilotes matériels sont chargés sous forme de **modules du noyau**. - Les outils `lspci`, `lsusb`, `lsblk` et `df` permettent d'identifier rapidement le matériel présent. - Savoir interpréter la mémoire disponible, l'espace disque et les périphériques est essentiel pour diagnostiquer un système. - **/proc** et **/sys** sont des arborescences **virtuelles**, mises à jour en temps réel par le noyau.

34. Observer les événements matériels

Objectif Lire les événements du noyau et observer la détection d'un périphérique.

Repères LPI 101 : 101.1, 101.2.

Commandes à connaître : `dmesg`, `journalctl -k`, `udevadm`, `lshw`, `hwinfo`.

Introduction

Linux détecte automatiquement le matériel lors du démarrage, mais aussi **dynamiquement** lorsqu'un périphérique est branché (USB, disque externe, carte réseau…). Ces événements sont gérés par le **sous-système udev** et enregistrés dans les **journaux du noyau**.

Ce chapitre aborde :

- l'affichage des **messages système du noyau** (`dmesg`) ;
- la gestion des périphériques par **udev** ;
- l'inspection matérielle avec des outils plus complets (`hwinfo`, `lshw`).

`dmesg` — messages du noyau

Le noyau Linux journalise tous les événements matériels : détection des disques, périphériques USB, pilotes chargés, erreurs, etc.

Afficher les derniers messages :

```
sudo dmesg | less
```

Chercher une ligne spécifique :

```
sudo dmesg | grep -i usb
sudo dmesg | grep -i ethernet
```

Afficher uniquement les nouveaux messages en temps réel (utile lors d'un branchement USB) :

```
sudo dmesg --follow
```

`journalctl -k` — journal des événements du noyau (systemd)

Sous systemd, les messages de `dmesg` sont aussi consultables via `journalctl` :

```
sudo journalctl -k
```

Afficher uniquement les 20 dernières lignes :

```
sudo journalctl -k -n 20
```

Surveiller en temps réel :

```
sudo journalctl -kf
```

`udevadm` — gestion des périphériques à chaud

`udev` est le composant chargé de **créer dynamiquement les fichiers dans** `/dev/` lors du branchement d'un périphérique.

Lister les périphériques connus par udev :

```
udevadm info --export-db | less
```

Afficher les informations d'un périphérique particulier :

```
udevadm info --query=all --name=/dev/sdb
```

Surveiller en direct les événements matériels (branchements / retraits) :

```
udevadm monitor
```

`hwinfo` — inventaire complet du matériel

`hwinfo` fournit un **résumé exhaustif** de la configuration matérielle (CPU, mémoire, réseau, bus, USB…).

Installer si nécessaire :

```
sudo apt install hwinfo
```

Lister tout le matériel détecté :

```
sudo hwinfo --short
```

Afficher les détails d'un composant précis :

```
sudo hwinfo --cpu
```

```
sudo hwinfo --network
```

`lshw` — description hiérarchique du matériel

`lshw` affiche la **hiérarchie complète du matériel**, depuis la carte mère jusqu'aux périphériques.

```
sudo lshw | less
```

Pour une vue synthétique :

```
sudo lshw -short
```

Exporter un rapport matériel (utile pour un inventaire) :

```
sudo lshw -html > rapport_hardware.html
```

À retenir

- Linux enregistre tous les événements matériels dans les **journaux du noyau**. - `dmesg` permet de suivre en direct ce que détecte le système (USB, disques, pilotes…). - Avec systemd, les messages du noyau sont aussi accessibles via `journalctl -k`. - **udev** gère la création dynamique des périphériques dans `/dev/` lors des branchements. - `udevadm monitor` permet d'observer en temps réel les événements matériels. - `hwinfo` fournit un inventaire matériel très complet. - `lshw` affiche une **vue hiérarchique** de l'ensemble du matériel du système.

35. Démarrage, arrêt et cibles systemd

Objectif Suivre le démarrage et reconnaître les principaux modes du système.

Repères LPI 101 : 101.2, 101.3.

Commandes à connaître : `systemctl`, `shutdown`, `wall`, `who`.

Introduction

Sous Linux, la gestion du démarrage, de l'arrêt et des modes de fonctionnement du système repose sur **systemd**, le gestionnaire d'initialisation utilisé par la plupart des distributions modernes. Il remplace les anciens systèmes comme **SysVinit** et permet de contrôler finement le démarrage des services, les transitions entre modes (console, graphique, secours) et l'arrêt propre du système.

LPI 101 / REPÈRE VISUEL

Du bouton de démarrage à la connexion

Un démarrage courant sur un PC Debian avec UEFI, GRUB et systemd.

1 **UEFI**
Le firmware initialise le matériel et lance le chargeur.

↓

2 **GRUB**
Le chargeur choisit et charge le noyau et l'initramfs.

↓

3 **Noyau + initramfs**
Le noyau démarre ; l'initramfs aide à accéder à la racine.

↓

4 **systemd · PID 1**
Sur la racine définitive, systemd organise les services.

Services disponibles
Réseau, SSH, journaux…

Connexion possible
Console ou interface graphique

systemd lance les unités selon leurs dépendances, souvent en parallèle.

Repère : firmware → chargeur → noyau → initialisation → système utilisable.

Les étapes d'un démarrage Debian courant : UEFI, GRUB, noyau et systemd.

Commandes de base

Arrêt et redémarrage avec systemctl

```
sudo systemctl poweroff
sudo systemctl reboot
```

Ces commandes demandent à **systemd** d'arrêter ou de redémarrer le système proprement. Tous les services sont stoppés dans le bon ordre avant la coupure ou le redémarrage.

Commande shutdown

La commande historique `shutdown` reste disponible et fonctionne toujours sous systemd. Elle permet en plus de **planifier un arrêt** ou un **redémarrage différé**.

```
sudo shutdown now         # arrêt immédiat
sudo shutdown -r now      # redémarrage immédiat
sudo shutdown +10         # arrêt dans 10 minutes
sudo shutdown -c          # annule un arrêt planifié
```

Différence entre poweroff et shutdown

Commande	Fonction principale	Particularité
`poweroff`	Arrêt immédiat du système	exécute la coupure directement via systemd
`shutdown`	Arrêt planifié ou différé	permet de choisir un délai, un redémarrage ou d'annuler l'action

En pratique, `shutdown` est plus flexible, tandis que `poweroff` agit plus directement.

Les cibles systemd (targets)

Systemd utilise des **targets** pour définir l'état du système (équivalent moderne des *runlevels*). Chaque target correspond à un ensemble de services actifs.

Target	Ancien runlevel	Description
`poweroff.target`	0	arrêt complet du système
`rescue.target`	1	mode de secours mono-utilisateur
`multi-user.target`	3	mode console multi-utilisateur
`graphical.target`	5	mode graphique
`reboot.target`	6	redémarrage du système

Commandes de gestion des modes

Afficher le mode par défaut :

```
systemctl get-default
```

Changer temporairement de mode :

```
sudo systemctl isolate multi-user.target
```

Définir le mode par défaut au démarrage :

```
sudo systemctl set-default graphical.target
```

Différence entre graphical.target et multi-user.target

`graphical.target`

Mode graphique complet — utilisé par défaut sur les distributions de bureau (Debian GNOME, Ubuntu, Fedora Workstation, etc.). Il lance :

- Tous les services nécessaires au multi-utilisateur (`multi-user.target`)
- Le **système d'affichage** (serveur Xorg ou compositeur Wayland)
- Le **gestionnaire de session** (GDM, LightDM, SDDM, etc.)
- La possibilité d'ouvrir une session graphique ; l'environnement de bureau est lancé lors de la connexion de l'utilisateur

Conséquences :

- Charge CPU et mémoire plus élevée
- Démarrage plus long
- Accès à une interface graphique complète

`multi-user.target`

Mode console / texte — sans interface graphique. Il charge uniquement les services essentiels : réseau, journaux, SSH, etc. L'utilisateur accède à une console texte (TTY) pour administrer le système.

Conséquences :

- Consommation mémoire et CPU faible
- Idéal pour les serveurs ou environnements légers
- Moins de risques de plantage liés à l'interface graphique

Mode de secours

Le **mode rescue** permet de démarrer un environnement minimal avec un seul utilisateur (root). Il est souvent utilisé pour la maintenance ou le dépannage.

Entrer dans le mode de secours :

```
sudo systemctl rescue
```

Revenir au mode normal :

```
sudo systemctl default
```

Relier les étapes du démarrage

```
BIOS/UEFI → chargeur GRUB → noyau et initramfs → init/systemd (PID 1)
          → services et cibles → connexion utilisateur
```

L'initramfs fournit un environnement initial, notamment pour accéder au système de fichiers racine. Les journaux du démarrage s'observent avec `journalctl -b` ; les messages du noyau avec `journalctl -k`.

Prévenir les utilisateurs

```
who
wall 'Redémarrage prévu dans 10 minutes pour maintenance.'
sudo shutdown -r +10
sudo shutdown -c
```

`wall` diffuse un message aux terminaux qui l'acceptent. `shutdown` annonce aussi l'arrêt planifié. Une transition de cible peut fermer des sessions : effectuer les démonstrations de secours depuis la console de la VM.

Repères historiques

SysVinit utilise des niveaux d'exécution, `/etc/inittab` et des scripts sous `/etc/init.d/`. `init` et `telinit` peuvent changer de niveau sur les systèmes concernés. Upstart est un autre système d'initialisation, à reconnaître sans le déployer ici. `acpid` traite des événements ACPI sur les installations qui l'utilisent ; systemd-logind prend aussi en charge des événements d'alimentation selon la configuration.

À retenir

* **systemd** gère le démarrage, l'arrêt et les différents modes du système. * Les **targets** remplacent les anciens *runlevels* (ex. `multi-user`, `graphical`, `rescue`). * `shutdown` permet de planifier ou d'annuler un arrêt, tandis que `poweroff` agit immédiatement. * `systemctl isolate` permet de basculer entre modes sans redémarrer. * Le **mode rescue** offre un environnement minimal pour effectuer des réparations système.

36. Gérer les services avec systemd

Objectif Distinguer état immédiat d'un service et activation au démarrage.

Repères LPI 101 : 101.2, 101.3.

Commandes à connaître : `systemctl`, `journalctl -u`.

Introduction

Sous Linux, les services (ou *daemons*) sont des processus qui s'exécutent en arrière-plan pour assurer des fonctions essentielles : réseau, SSH, serveur web, journalisation, etc. Depuis l'adoption de **systemd**, la gestion de ces services est unifiée grâce à la commande **systemctl**, qui permet de les démarrer, arrêter, activer au démarrage ou vérifier leur état.

Principes de base

Un **service** est généralement défini par une unité (*unit*) située dans `/etc/systemd/system/` ou `/lib/systemd/system/`. Ces fichiers décrivent comment lancer, arrêter et surveiller le service.

Les services actifs en arrière-plan portent souvent un nom terminé par `.service`, par exemple :

- `ssh.service` : service SSH
- `apache2.service` : serveur web Apache
- `cron.service` : planificateur de tâches

Commandes principales

Action	Commande	Description
Démarrer un service	`sudo systemctl start ssh`	Lance le service immédiatement
Arrêter un service	`sudo systemctl stop ssh`	Stoppe le service sans le désactiver
Redémarrer un service	`sudo systemctl restart ssh`	Redémarre le service (utile après modification)
Recharger la configuration	`sudo systemctl reload ssh`	Recharge la configuration sans interrompre le service
Vérifier l'état	`systemctl status ssh`	Affiche l'état du service et les journaux récents
Lister les services actifs	`systemctl list-units --type=service --state=running`	Montre les services en cours d'exécution

---

Activation et désactivation au démarrage

Les services peuvent être configurés pour se lancer automatiquement au démarrage du système.

```
sudo systemctl enable ssh
sudo systemctl disable ssh
```

- `enable` → crée les liens prévus par la section `[Install]`, par exemple dans `multi-user.target.wants/`
- `disable` → retire les liens d'activation concernés, sans arrêter le service à lui seul

Vérifier si un service est activé au démarrage :

```
systemctl is-enabled ssh
```

Vérification et dépannage

Lister les services en échec :

```
systemctl --failed
```

Afficher uniquement les services inactifs :

```
systemctl list-units --state=inactive --type=service
```

Consulter les journaux récents d’un service :

```
journalctl -u ssh
```

Suivre les journaux en temps réel :

```
journalctl -u ssh -f
```

Exemple d’utilisation pratique

```
# Démarrer et vérifier le service SSH
sudo systemctl start ssh
systemctl status ssh

# Lancer Apache et l’activer au démarrage
sudo systemctl start apache2
sudo systemctl enable apache2

# Recharger la configuration sans couper le service
sudo systemctl reload apache2
```

Structure d’un fichier de service

Exemple simplifié d’un fichier administrateur `/etc/systemd/system/monservice.service` :

```
[Unit]
Description=Mon service personnalisé
After=network.target

[Service]
ExecStart=/usr/local/bin/script.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Chaque section a un rôle :

- `[Unit]` → dépendances et description
- `[Service]` → commande à exécuter et comportement
- `[Install]` → cibles dans lesquelles le service est activé

Actif et activé ne veulent pas dire la même chose

`start` agit maintenant ; `enable` prépare l'activation selon les dépendances au démarrage. `sudo systemctl enable --now ssh` combine activation et démarrage. Après création ou modification du fichier d'unité, utiliser `sudo systemctl daemon-reload`. `reload` concerne la configuration de l'application et n'est disponible que si le service le prévoit.

À retenir

- **systemd** gère les services via la commande **systemctl**.
- Un service est une *unit* décrite dans `/etc/systemd/system/` ou `/lib/systemd/system/`.
- Les commandes principales :
- `start`, `stop`, `restart`, `reload`, `status`
- `enable`, `disable`, `is-enabled`
- Les journaux des services sont accessibles avec **journalctl -u nom_du_service**.
- La commande `systemctl --failed` permet d'identifier rapidement les services en erreur.
- Comprendre et gérer les services est essentiel pour administrer un système Linux, qu'il soit serveur ou poste de travail.

37. Consulter les journaux et diagnostiquer

Objectif Choisir le journal utile et filtrer ses messages.

Repères LPI 101 : 101.2.

Commandes à connaître : `journalctl`, `less`, `tail`, `grep`.

Introduction

Sous Linux, l'analyse des **journaux (logs)** est essentielle pour comprendre le fonctionnement du système, diagnostiquer des pannes et surveiller les services. Depuis l'arrivée de **systemd**, la majorité des logs sont centralisés par **systemd-journald** et consultables avec `journalctl`. Les applications traditionnelles continuent néanmoins d'écrire leurs fichiers de logs dans `/var/log/`.

Ce chapitre couvre :

- l'utilisation efficace de **journalctl** ;
- la recherche avancée via les champs du journal systemd ;
- l'utilisation des **facilities** et **priorités** ;
- les fichiers de logs classiques dans `/var/log/`.

Logs centralisés avec journalctl

`journalctl` permet d'explorer les journaux collectés par **systemd-journald**.

Affichage général

```
sudo journalctl           # Tous les logs accessibles
sudo journalctl -r        # Affichage inversé (du plus récent au plus ancien)
sudo journalctl -b        # Logs du dernier démarrage
sudo journalctl -f        # Suivi en temps réel
```

Filtrer par gravité (priorité)

```
sudo journalctl -p 3 -xb   # Erreurs et niveaux critiques
sudo journalctl -p warning # Avertissements et niveaux plus graves
sudo journalctl -p alert..err
```

Filtrer par service (unit systemd)

```
sudo journalctl -u ssh
sudo journalctl -u apache2
```

Filtrer par date

```
sudo journalctl --since "2025-01-01"
sudo journalctl --since "1 hour ago"
sudo journalctl --until "10 minutes ago"
```

Filtrer par application ou champ structuré

`journalctl` peut filtrer sur n'importe quel **champ structuré** du journal : (exemples issus du manuel `systemd.journal-fields`)

```
journalctl _PID=1234
journalctl _EXE=/usr/bin/ssh
journalctl _COMM=nginx
journalctl _SYSTEMD_UNIT=cron.service
```

Facilities et priorités syslog

Avant systemd, le système de logs reposait principalement sur **syslog**. Systemd conserve ces informations pour la compatibilité.

Liste des facilities

Code	Facility	Description
0	kern	Messages du noyau
1	user	Programmes utilisateur
2	mail	Serveurs de mail
3	daemon	Services système
4	auth	Authentification
5	syslog	Messages du démon syslog
6	lpr	Impression
7	news	Usenet (ancien)
8	uucp	Protocole UUCP
9	cron	Planification de tâches
10	authpriv	Authentification privée
11	ftp	Serveur FTP
12	ntp	Synchronisation NTP
13	security	Sécurité
14	console	Console locale
15	cron	Tâches planifiées
16–23	local0–local7	Facilités réservées

Filtrer par facility dans journalctl

Afficher l’aide :

```
journalctl --facility=help
```

Filtrer :

```
journalctl --facility daemon
```

Priorités (gravité)

Code	Nom	Signification
0	emerg	Système inutilisable
1	alert	Action immédiate
2	crit	Erreur critique
3	err	Erreur
4	warning	Avertissement
5	notice	Info importante
6	info	Information
7	debug	Débogage

Exemple :

```
journalctl -p alert..err
```

Fichiers de logs classiques dans /var/log

Certains programmes n'utilisent pas systemd-journald et continuent d'écrire dans `/var/log/`.

Fichier	Description
`/var/log/syslog` ou `/var/log/messages`	Messages généraux
`/var/log/auth.log`	Authentification (sudo, ssh)
`/var/log/kern.log`	Messages du noyau
`/var/log/dpkg.log`	Actions sur les paquets Debian
`/var/log/apt/history.log`	Historique APT
`/var/log/apache2/*`	Logs d'Apache
`/var/log/cron.log`	Tâches cron
`/var/log/Xorg.0.log`	Serveur graphique

Commandes utiles

```
sudo tail -n 30 /var/log/syslog
sudo less /var/log/auth.log
sudo grep "error" /var/log/syslog
```

Pourquoi la migration vers journalctl / systemd-journald ?

- Le journal de systemd collecte des **logs structurés et indexés**, avec champs précis (PID, unité systemd, exécutable, etc.), ce qui facilite les filtrages et recherches.
- Il centralise dans un même format les logs du noyau, des services, et des applications, évitant de fouiller dans de nombreux fichiers dispersés.
- Le format binaire (ou semi-binaire) permet d'optimiser la recherche et de produire différents formats de sortie (JSON, export, etc.).
- Il permet une gestion intégrée de la rotation, de la persistance ou non des logs (en mémoire ou sur disque), et des quotas de taille.
- Pour la sécurité et la conformité, systemd-journald offre des métadonnées supplémentaires et des fonctions de scellage (log sealing) pour garantir l'intégrité des logs.

Disponibilité des fichiers de journaux

Les fichiers `/var/log/syslog`, `/var/log/auth.log` et `/var/log/cron.log` dépendent des services et règles de journalisation installés. Sur une VM sans rsyslog, certains peuvent être absents : utiliser `journalctl` et vérifier la configuration plutôt que supposer une panne. Les fichiers de journaux doivent être lus avec les droits nécessaires.

À retenir

- `journalctl` centralise la majorité des logs du système et des services. - Les **facilities** et **priorités** permettent de classer les messages par rôle et gravité. - Les fichiers sous `/var/log/` restent incontournables pour certains programmes traditionnels. - La commande `journalctl -u <service>` est essentielle pour diagnostiquer un service. - `--since`, `--until`, `-p`, `-u`, `-f` sont les options les plus utilisées pour l'analyse quotidienne.

38. Comprendre GRUB et la récupération

Objectif Lire la configuration de GRUB et comprendre les options de récupération.

Repères LPI 101 : 101.2, 102.2.

Introduction

GRUB (GRand Unified Bootloader) est le chargeur d'amorçage utilisé par la majorité des distributions Linux. Il est responsable de :

- charger le noyau Linux ;
- transmettre les paramètres au noyau ;
- proposer un menu de démarrage ;
- permettre la récupération en cas de problème (mode rescue ou édition des options).

Comprendre GRUB est essentiel pour diagnostiquer un système qui ne démarre plus, modifier les paramètres du noyau, ou restaurer un accès administrateur local dans un laboratoire autorisé.

Fonctionnement général de GRUB

GRUB s'affiche juste après le BIOS/UEFI. Il accède à sa configuration dans :

```
/boot/grub/grub.cfg
```

Ce fichier est **généré automatiquement** depuis les scripts dans `/etc/grub.d/` et les options de `/etc/default/grub`.

Pour régénérer la configuration (ex : après modification du noyau ou du fond d'écran) :

```
sudo update-grub
```

Navigation dans le menu GRUB

Lors du démarrage, GRUB affiche un menu contenant :

- l'entrée principale (Debian / Ubuntu / autre) ;

• le sous-menu "Options avancées" ;

• les différents noyaux installés.

Touches utiles :

Touche	Action
Entrée	Démarrer
e	Éditer les options de démarrage
c	Ouvrir un shell GRUB
Esc	Revenir au menu principal

Modifier les options du noyau (édition rapide)

Utile pour :

• activer des options de debug ;

• désactiver un pilote problématique ;

• démarrer en mode single-user ;

• réinitialiser un mot de passe root (procédure dans le TP d'origine).

Étapes :

1. Sélectionner l'entrée Debian/Ubuntu.
2. Appuyer sur **e** pour éditer.
3. Trouver la ligne commençant par : `text linux /boot/vmlinuz-...`
4. Ajouter une option à la fin, par exemple : `text systemd.unit=rescue.target`
5. Démarrer avec **Ctrl + X** ou **F10**.

Configuration du fichier /etc/default/grub

Ce fichier contient les options principales :

```
GRUB_DEFAULT=0
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR=`( . /etc/os-release && echo ${NAME} )`
GRUB_CMDLINE_LINUX_DEFAULT="quiet"
GRUB_CMDLINE_LINUX=""
```

- **GRUB_DEFAULT=0** Sélectionne automatiquement **la première entrée** du menu GRUB au démarrage (index 0).
- **GRUB_TIMEOUT=5** Délai (en secondes) durant lequel GRUB attend avant de démarrer l'entrée par défaut.
- **GRUB_DISTRIBUTOR=**`( . /etc/os-release && echo ${NAME} )` Automatiquement remplacé par le **nom de la distribution** (Debian, Ubuntu…), utilisé pour étiqueter les entrées dans le menu GRUB.
- **GRUB_CMDLINE_LINUX_DEFAULT="quiet"** Options passées au noyau en mode normal. `quiet` masque les messages de démarrage pour un boot plus propre (souvent ajouté : `quiet splash`).
- **GRUB_CMDLINE_LINUX=""** Options supplémentaires toujours passées au noyau (souvent laissé vide, utilisé pour des réglages avancés).

Une fois modifié :

```
sudo update-grub
```

Reconnaître les variantes de GRUB

GRUB 2 utilise `grub.cfg`, généré à partir de la configuration. GRUB Legacy utilisait notamment `menu.lst` ou `grub.conf`. Sous Debian, `update-grub` facilite la génération ; `grub-mkconfig` est l'outil sous-jacent à reconnaître. `grub-install` installe des éléments du chargeur : ses arguments dépendent du mode BIOS/UEFI et de la cible.

En récupération, `rescue.target` et `emergency.target` offrent des environnements de secours distincts ; `init=/bin/bash` remplace le processus initial par un shell. Le chiffrement du disque et les protections de démarrage peuvent imposer d'autres étapes.

À retenir — GRUB

- GRUB charge le noyau et permet de modifier les paramètres du démarrage. - L'édition rapide (touche **e**) est indispensable pour dépanner un système. - Les modes **rescue**, **emergency** et `init=/bin/bash` permettent différents niveaux de récupération. - `/etc/default/grub` contrôle la configuration principale, et `update-grub` la régénère. - La réinitialisation du mot de passe root via GRUB nécessite un accès local à la machine.

39. Se connecter à distance avec SSH

Objectif Utiliser SSH, transférer un fichier et comprendre l'authentification par clé.

Commandes à connaître : `ssh`, `ssh-keygen`, `ssh-copy-id`, `scp`.

Introduction

SSH (Secure Shell) est un protocole permettant d'administrer un système Linux à distance, de manière chiffrée. Il permet :

- d'exécuter des commandes à distance,
- de transférer des fichiers,
- ou de se connecter à une session shell distante.

Sous Linux, le client SSH est intégré. Sous Windows, il est inclus depuis Windows 10 et peut être utilisé via **PowerShell** ou **Windows Terminal**.

Liens externes :

- Tutoriel SSH — Microsoft Learn
- Apprenez à utiliser le client SSH de Windows 11

Utilisation du client SSH

Syntaxe générale :

```
ssh [-p port] [utilisateur@]hôte
```

Exemples :

```
ssh debian@192.168.56.101
ssh root@serveur.exemple.local -p 2222
```

Si aucun utilisateur n'est précisé, celui de la session actuelle est utilisé. Lors de la première connexion, vérifier l'empreinte de la clé d'hôte auprès d'une source connue avant de l'accepter.

Génération de clés SSH

Une paire de clés est composée :

- d'une **clé privée** (gardée sur le poste local),
- et d'une **clé publique** (copiée sur le serveur).

Créer une paire de clés sur le poste client :

```
ssh-keygen -t ed25519
```

Le programme crée :

```
~/.ssh/id_ed25519      (clé privée)
~/.ssh/id_ed25519.pub  (clé publique)
```

Ed25519 est un choix courant pour ce laboratoire ; les algorithmes autorisés dépendent aussi des politiques et des systèmes utilisés.

Copier la clé sur le serveur

Depuis Linux

```
ssh-copy-id utilisateur@serveur
```

Depuis Windows (PowerShell)

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh utilisateur@serveur "mkdir -p ~/.ssh && cat
     >> ~/.ssh/authorized_keys"
```

Vérifie ensuite que le contenu de la clé publique est bien ajouté dans `~/.ssh/authorized_keys` sur le serveur.

Connexion sans mot de passe

Une fois la clé installée :

```
ssh utilisateur@serveur
```

Le mot de passe du compte distant n'est normalement plus demandé. Une **phrase de passe de la clé privée** peut encore être demandée si la clé est protégée et n'est pas chargée dans un agent.

Après avoir vérifié l'authentification par clé dans **une seconde connexion** et conservé un accès console, on peut désactiver le mot de passe dans une configuration du serveur :

```
sudo tee /etc/ssh/sshd_config.d/00-disable-password.conf <<< "PasswordAuthentication no"
sudo sshd -t
sudo systemctl reload ssh
```

Commandes utiles

Action	Commande
Tester une connexion	`ssh utilisateur@serveur`
Copier un fichier distant	`scp utilisateur@serveur:/chemin/fichier .`
Copier un répertoire entier	`scp -r dossier utilisateur@serveur:/chemin/`
Vérifier le service SSH	`sudo systemctl status ssh`
Voir les logs SSH	`sudo journalctl -u ssh`

Exemple de fichier de configuration client (`~/.ssh/config`)

```
Host debian
    HostName 192.168.56.101
    User sam
    Port 22
    IdentityFile ~/.ssh/id_ed25519
```

Connexion simplifiée :

```
ssh debian
```

Structure du dossier `.ssh` sous Windows

Sous Windows, les fichiers SSH sont stockés dans : `%USERPROFILE%\.ssh\` (ou `$env:USERPROFILE\.ssh` dans PowerShell)

Fichier	Rôle
`id_ed25519` ou `id_rsa`	Clé privée (à ne jamais partager).

Fichier	Rôle
`id_ed25519.pub` ou `id_rsa.pub`	Clé publique à copier sur le serveur.
`known_hosts`	Liste des serveurs déjà connus et validés.
`config`	Fichier de configuration du client SSH (alias, ports, chemins de clés, etc.).
`known_hosts.old`	Sauvegarde automatique de la liste des serveurs connus.

> Ces fichiers sont automatiquement utilisés par **OpenSSH**, intégré à PowerShell et Windows Terminal.

Bonnes pratiques sur la gestion des clés

Il est **possible d'utiliser la même clé SSH sur plusieurs serveurs**, mais ce n'est pas toujours conseillé.

Cas d'usage	Recommandation
Environnement personnel / lab	Une seule clé pour tout est acceptable.
Environnement professionnel	Créer une clé par utilisateur ou par serveur.
Serveurs sensibles (production)	Clé spécifique par utilisateur, protégée par passphrase.
Automatisation (Ansible, scripts, CI/CD)	Clé dédiée à la tâche, droits restreints et stockage protégé ; utiliser un agent ou un gestionnaire de secrets selon le contexte.

Exemple de génération de deux clés :

```
# Clé pour le lab
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_lab

# Clé pour la production
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_prod
```

Puis configuration dans `~/.ssh/config` :

```
Host lab-*
    IdentityFile ~/.ssh/id_ed25519_lab

Host prod-*
    IdentityFile ~/.ssh/id_ed25519_prod
```

À retenir

- SSH chiffre les communications entre client et serveur.

- Une clé privée protégée et une vérification de l'identité du serveur renforcent l'authentification.
- Les fichiers de configuration et de clés se trouvent dans `~/.ssh/` (ou `%userprofile%\.ssh\` sous Windows).
- SCP permet de transférer des fichiers de manière sécurisée.
- Une clé peut être utilisée sur plusieurs serveurs, mais il est préférable d'en générer une par usage critique.

C01. Donner des droits précis avec les ACL

Objectif Ajouter une autorisation ciblée après avoir compris les permissions classiques.
Complément À découvrir après les bases, selon le rythme du groupe.

Introduction

Les **ACL** (*Access Control Lists*) permettent d'aller **au-delà des permissions classiques** (`rwx` pour propriétaire, groupe et autres). Elles offrent une **gestion plus fine des droits** d'accès, en autorisant des permissions spécifiques pour **plusieurs utilisateurs ou groupes** sur un même fichier ou répertoire.

Cette section explique :

- comment **installer** et **afficher** les ACL ;
- comment **attribuer** des droits avancés à des utilisateurs et groupes ;
- comment **comprendre le fonctionnement du masque** (*mask*).

Installation du support ACL

Les systèmes modernes supportent les ACL nativement. Si les commandes `getfacl` et `setfacl` ne sont pas disponibles, installez le paquet suivant :

```
sudo apt install acl
```

Commandes principales

`getfacl` — afficher les ACL

Cette commande affiche les permissions étendues d'un fichier ou répertoire.

```
getfacl fichier.txt
```

Exemple de sortie :

```
# file: fichier.txt
# owner: sam
```

```
# group: compta
user::rw-
group::r--
group:rh:r-x
mask::rwx
other::r--
```

Explications :

- `user::` → droits du propriétaire
- `group::` → droits du groupe principal
- `group:rh:` → droits spécifiques au groupe `rh`
- `mask::` → limite maximale des permissions effectives
- `other::` → droits des autres utilisateurs

`setfacl` — définir une ACL

Attribuer un droit spécifique à un utilisateur :

```
setfacl -m u:alice:rw fichier.txt
```

Attribuer un droit spécifique à un groupe :

```
setfacl -m g:rh:r-x fichier.txt
```

Supprimer une ACL :

```
setfacl -x u:alice fichier.txt
```

Héritage et ACL par défaut

Les répertoires peuvent définir des **ACL par défaut** : elles seront automatiquement appliquées aux nouveaux fichiers ou dossiers créés à l'intérieur.

Définir une ACL par défaut :

```
setfacl -m d:u:alice:rwx /srv/partage
```

Afficher les ACL par défaut :

```
getfacl /srv/partage
```

Les lignes commençant par `default:` indiquent ces ACL héritées.

Le rôle du masque (`mask`)

Le **mask** limite les droits effectifs des entrées ACL pour les utilisateurs et groupes **autres que le propriétaire**. Même si une ACL accorde `rwx`, le **mask** peut restreindre l'accès réel.

Vérifier l'effet du masque :

```
getfacl fichier.txt
```

Modifier le masque :

```
setfacl -m m:r-x fichier.txt
```

Supprimer toutes les ACL

Pour réinitialiser un fichier à ses permissions classiques :

```
setfacl -b fichier.txt
```

(`-b` supprime toutes les ACL associées)

À retenir

- Les **ACL** permettent une **gestion plus granulaire** que les permissions classiques `rwx`.

- `getfacl` affiche les ACL, `setfacl` les modifie.
- Le **mask** limite les droits effectifs des ACL.
- Les **ACL par défaut** permettent l'héritage dans un répertoire.
- `setfacl -b` réinitialise un fichier en supprimant toutes les ACL.

C02. Comprendre et manipuler LVM

Objectif Relier volumes physiques, groupes et volumes logiques.

Repères LPI 101 : 102.1.

Complément À découvrir après les bases, selon le rythme du groupe.

Introduction

Le **LVM (Logical Volume Manager)** est un système de gestion de volumes logiques. Il permet d'allouer, redimensionner et gérer l'espace disque de manière **beaucoup plus souple** que les partitions classiques.

LVM est particulièrement utile sur les serveurs ou les environnements virtualisés, car il permet :

- d'**agrandir** des volumes et certains systèmes de fichiers à chaud ; une réduction nécessite une procédure propre au système de fichiers ;
- de **regrouper plusieurs disques** physiques dans un seul espace logique ;
- de **créer des snapshots** (états à un instant donné) qui peuvent aider à réaliser des sauvegardes.

Structure logique du LVM

LVM repose sur trois couches principales :

```
Disques physiques → Physical Volumes (PV)
         ↓
Groupes de volumes → Volume Groups (VG)
         ↓
Volumes logiques → Logical Volumes (LV)
```

Élément	Description	Exemple
PV (Physical Volume)	Disque ou partition utilisée par LVM	`/dev/sdb1`
VG (Volume Group)	Ensemble de PV formant un "pool" d'espace	`vg_data`
LV (Logical Volume)	Partition logique que l'on formate et monte	`/dev/vg_data/lv_backup`

Étapes de création d’un volume LVM

□□ Réaliser ces commandes **sur un disque de test uniquement** (ex. `/dev/sdb` ajouté dans VirtualBox).

Créer un volume physique (PV)

```
sudo pvcreate /dev/sdb1
```

Afficher les volumes physiques existants :

```
sudo pvs
sudo pvdisplay
```

Créer un groupe de volumes (VG)

```
sudo vgcreate vg_data /dev/sdb1
```

Afficher les groupes de volumes :

```
sudo vgs
sudo vgdisplay
```

Créer un volume logique (LV)

```
sudo lvcreate -L 1G -n lv_backup vg_data
```

- `-L 1G` → taille du volume logique
- `-n lv_backup` → nom du volume
- `vg_data` → nom du groupe de volumes

Lister les volumes logiques :

```
sudo lvs
sudo lvdisplay
```

Formater et monter le volume

Formater en ext4 :

```
sudo mkfs.ext4 /dev/vg_data/lv_backup
```

Créer un point de montage et monter :

```
sudo mkdir /mnt/lvm
sudo mount /dev/vg_data/lv_backup /mnt/lvm
```

Vérifier :

```
lsblk
df -h /mnt/lvm
```

Agrandir un volume logique (exemple)

Si le groupe contient encore de l'espace libre :

```
sudo lvextend -L +500M /dev/vg_data/lv_backup
sudo resize2fs /dev/vg_data/lv_backup
```

La première commande agrandit le volume logique ; la seconde agrandit explicitement son système de fichiers ext4. XFS utilise d'autres outils et ne permet pas la réduction de son système de fichiers.

Supprimer proprement les volumes (pour test uniquement)

```
sudo umount /mnt/lvm
sudo lvremove /dev/vg_data/lv_backup
sudo vgremove vg_data
sudo pvremove /dev/sdb1
```

Visualisation synthétique

```
sudo pvs   # liste les volumes physiques
sudo vgs   # liste les groupes de volumes
sudo lvs   # liste les volumes logiques
```

Avantages de LVM

- Redimensionnement dynamique (agrandissement à chaud possible).

• Agrégation de plusieurs disques dans un seul espace.

• Création de **snapshots** pour sauvegardes rapides.

• Gestion simplifiée de l’espace de stockage dans le temps.

À retenir

* **LVM** offre une gestion flexible du stockage, plus puissante que les partitions classiques. * Trois niveaux hiérarchiques :

• **PV** → disques physiques utilisés par LVM

• **VG** → groupe de volumes, regroupe un ou plusieurs PV

• **LV** → volumes logiques, montés comme des partitions normales

• Commandes essentielles :

• `pvcreate`, `vgcreate`, `lvcreate` → création

• `pvs`, `vgs`, `lvs` → affichage

• `lvextend` + `resize2fs` → agrandissement

• `lvremove`, `vgremove`, `pvremove` → suppression

• LVM est idéal sur **serveur ou machine virtuelle**, mais à manipuler uniquement sur un **disque de test** lors des premières expérimentations.

C03. Écrire un premier script Bash

Objectif Regrouper quelques commandes et utiliser des paramètres.

Complément À découvrir après les bases, selon le rythme du groupe.

Qu’est-ce qu’un script shell ?

Un **script shell** est un fichier texte contenant une suite de commandes exécutées automatiquement par l’interpréteur **bash** (ou un autre shell). Il permet d’automatiser des tâches répétitives : sauvegardes, vérifications, installations, etc.

Exécution et droits

Pour une exécution directe, un script utilise généralement une **ligne d’en-tête** appelée *shebang*, qui indique quel interpréteur doit l’exécuter :

```
#!/bin/bash
```

Pour qu’un script soit exécutable :

```
chmod +x monscript.sh
./monscript.sh
```

Variables et expansions

Les variables stockent des valeurs temporaires utilisées dans le script.

```
NOM="Sam"
echo "Bonjour $NOM"
```

Quelques variables prédéfinies utiles :

- `$HOME` → répertoire personnel
- `$USER` → nom de l’utilisateur courant
- `$PWD` → répertoire courant
- `$0`, `$1`, `$2` → nom du script et paramètres passés en ligne de commande

---

□ À retenir

- Un **script shell** est un simple fichier texte exécuté automatiquement par le shell. - Le *shebang* (`#!/bin/bash`) indique quel interpréteur utiliser. - Pour `./script.sh`, le droit d'exécution est nécessaire ; `bash script.sh` permet aussi de lancer un script lisible sans ce droit. - Les **variables** permettent de stocker des valeurs et de les réutiliser dans le script. - Les variables prédéfinies comme `$HOME`, `$USER`, `$PWD` ou `$1` sont très utiles dans les scripts. - L'exécution d'un script peut se faire avec `./script.sh` depuis son répertoire.

Passer un paramètre

```
#!/bin/bash
nom="${1:-Sam}"
printf 'Bonjour %s\n' "$nom"
```

Enregistrer dans `bonjour.sh`, puis lancer `bash bonjour.sh Alex`. `$1` représente le premier argument et `${1:-Sam}` fournit une valeur par défaut.

C04. Découvrir les conditions et les boucles

Objectif Lire un test simple et répéter une action.
Complément À découvrir après les bases, selon le rythme du groupe.

Les tests conditionnels

```
if [ -f /etc/passwd ]; then
  echo "Le fichier existe."
else
  echo "Fichier introuvable."
fi
```

Autres opérateurs utiles :

- `-d` → test si c'est un dossier
- `-e` → test si le fichier existe
- `-z` → test si une variable est vide
- `=` ou `!=` → comparaison de chaînes
- `-lt`, `-gt`, `-eq` → comparaisons numériques

Les boucles

for — répète une commande pour chaque élément :

```
for i in 1 2 3; do
  echo "Tour n°$i"
done
```

while — répète tant qu'une condition est vraie :

```
n=1
while [ $n -le 3 ]; do
  echo "Itération $n"
  n=$((n+1))
done
```

Commandes interactives

Permettent de créer des scripts dynamiques :

```
read -p "Quel est ton nom ? " nom
echo "Bonjour $nom !"
```

`echo -e` permet d'interpréter les caractères spéciaux :

```
echo -e "Ligne 1\nLigne 2"
```

À retenir

- Les tests conditionnels permettent d'exécuter du code selon une condition (`if`, `else`). - Les opérateurs `-f`, `-d`, `-e`, `-z`, `=`, `!=`, `-lt`, `-gt`, `-eq` servent à comparer fichiers, chaînes et valeurs. - Les boucles `for` et `while` permettent de répéter des instructions automatiquement. - `read` permet de demander une entrée à l'utilisateur dans un script. - `echo -e` active l'interprétation des caractères spéciaux comme `\n`.

C05. Lire des scripts utilitaires

Objectif Comprendre une petite automatisation et vérifier son résultat.

Complément À découvrir après les bases, selon le rythme du groupe.

Quelques exemples simples :

Afficher des infos système :

```
#!/bin/bash
echo "Utilisateur : $USER"
echo "Répertoire : $PWD"
echo "Système actif depuis :"
uptime
```

Sauvegarde basique :

```
#!/bin/bash
SRC="/etc"
DEST="/mnt/backup"
DATE=$(date +%Y-%m-%d)
if tar czf "$DEST/sauvegarde-$DATE.tar.gz" "$SRC"; then
  echo "Sauvegarde terminée."
else
  echo "Échec de la sauvegarde." >&2
  exit 1
fi
```

À retenir

* Un **script shell** est une suite de commandes exécutées automatiquement. * Il commence par un **shebang** (`#!/bin/bash`). * On doit lui donner le **droit d'exécution** avec `chmod +x`. * Les **variables** stockent et réutilisent des valeurs. * Les **tests** (`if`, `else`) et **boucles** (`for`, `while`) contrôlent le déroulement du script. * `read` et `echo` rendent les scripts interactifs. * Même avec des bases simples, on peut automatiser beaucoup de tâches répétitives.

Préparer la destination L'exemple de sauvegarde suppose que `/mnt/backup` existe, est monté si nécessaire et est accessible en écriture. Vérifier la destination et les droits de lecture de la source avant l'exécution.

C06. Découvrir Linux sous Windows avec WSL

Objectif Reconnaître les possibilités de WSL et ses différences avec la VM de formation.

Complément À découvrir après les bases, selon le rythme du groupe.

Introduction

WSL (Windows Subsystem for Linux) permet d'exécuter une distribution Linux directement sous Windows, avec une intégration forte à Windows ; WSL 2 utilise une machine virtuelle légère gérée par Windows. C'est une solution légère et intégrée qui offre un véritable environnement Linux pour le développement, l'administration ou la formation.

Sources officielles :

- Documentation officielle Microsoft WSL
- Tutoriels WSL sur IT-Connect

Présentation générale

WSL offre une compatibilité entre le noyau Windows et Linux. Deux versions existent :

Version	Description
WSL 1	Traduit les appels système Linux vers le noyau Windows. Plus rapide pour les opérations sur fichiers Windows.
WSL 2	Utilise un vrai noyau Linux dans une machine virtuelle optimisée. Large compatibilité des appels système ; réseau selon la configuration Windows/WSL.

Sur une version compatible de Windows, `wsl --install` permet d'installer les composants nécessaires. WSL n'est pas nécessairement déjà installé sur le poste.

Installation de WSL

Ouvre **PowerShell en administrateur** et lance :

```
wsl --install
```

Cette commande :

- active les composants nécessaires ;
- installe WSL 2 ;
- télécharge la dernière distribution Ubuntu.

Vérifie ensuite l'état :

```
wsl --status
```

Attention à la compatibilité avec VirtualBox

Depuis **WSL 2**, Windows active la technologie de virtualisation **Hyper-V**. Or, **VirtualBox** et **Hyper-V** utilisent toutes deux la même couche de virtualisation matérielle (VT-x/AMD-V). Cela peut provoquer des lenteurs importantes, voire empêcher le démarrage des machines VirtualBox.

Gestion des distributions

Lister les distributions disponibles :

```
wsl --list --online
```

Installer une distribution spécifique :

```
wsl --install -d Debian
```

Lister les distributions installées :

```
wsl --list --verbose
```

Exemple :

```
  NAME      STATE           VERSION
* Ubuntu    Running         2
  Debian    Stopped         2
```

On peut aussi passer par le Microsoft store pour installer une distribution linux

Commandes de gestion

Action	Commande
Démarrer une distribution	`wsl -d Debian`
Changer la version	`wsl --set-version Debian 2`
Définir WSL 2 par défaut	`wsl --set-default-version 2`
Arrêter toutes les distributions	`wsl --shutdown`
Exporter une distribution	`wsl --export Debian D:\Sauvegarde\debian.tar`
Importer une sauvegarde	`wsl --import Debian D:\WSL\Debian D:\Sauvegarde\debian.tar --version 2`

Accès aux fichiers

Sous WSL, les disques Windows sont montés automatiquement dans `/mnt/` :

Disque Windows	Emplacement sous Linux
`C:\`	`/mnt/c/`
`D:\`	`/mnt/d/`

Exemple :

```
cd /mnt/c/Users
ls
```

Pour accéder aux fichiers Linux depuis Windows, utiliser `explorer.exe .` depuis WSL ou le partage `\\wsl$` / `\\wsl.localhost`. Ne pas modifier directement les fichiers internes du disque virtuel de la distribution dans les dossiers système de Windows.

Intégration Windows ↔ Linux

Fonction	Exemple
Exécuter une commande Linux depuis PowerShell	`wsl ls -l /home`
Exécuter une commande Windows depuis Linux	`explorer.exe .`
Ouvrir VS Code dans le dossier courant	`code .`
Partage du presse-papier et du réseau	Automatique sous WSL 2

Gestion des ressources

Les ressources utilisées par WSL 2 peuvent être ajustées avec le fichier de configuration suivant :

Chemin : `C:\Users\<nom>\.wslconfig`

Exemple :

```
[wsl2]
memory=4GB
processors=2
swap=0
```

Appliquer les changements :

```
wsl --shutdown
```

> Il y a une interface graphique `WSL Settings`

Bonnes pratiques

- Préférer **WSL 2** pour une large compatibilité avec les outils Linux.
- Accéder aux fichiers Linux par les mécanismes prévus : `explorer.exe .` ou le partage WSL.
- Utiliser WSL pour les tests et scripts ; garder la VM Debian commune pour les exercices de démarrage et de stockage.
- Choisir WSL, une VM ou un serveur selon la tâche ; ils ne présentent pas tous le même matériel ni le même démarrage.

À retenir

- WSL offre un environnement Linux complet directement dans Windows.

- WSL 2 repose sur un vrai noyau Linux et peut utiliser systemd et des outils de conteneurs, selon les versions et la configuration.
- Les fichiers Windows sont accessibles sous `/mnt/`.
- Idéal pour pratiquer Linux sans installation complète.

C07. Découvrir Cloud-init

Objectif Comprendre l'initialisation d'une image cloud et lire un exemple YAML.

Repères LPI 101 : 102.6.

Complément À découvrir après les bases, selon le rythme du groupe.

Introduction

Cloud-init est un outil utilisé dans les environnements **cloud** (AWS, Azure, OpenStack, Proxmox, VirtualBox, etc.) pour **personnaliser automatiquement une machine virtuelle lors de son premier démarrage**. Il permet d'automatiser la configuration d'une instance sans intervention manuelle : création d'utilisateurs, configuration réseau, installation de paquets, déploiement de clés SSH, etc.

Cloud-init récupère les métadonnées et la configuration auprès d'une **source de données** (datasource), en général au format **YAML**, et exécute les actions correspondantes dès le premier boot de la machine. C'est un **outil largement utilisé** pour la configuration initiale des systèmes Linux déployés à partir d'images "cloud".

Fonctionnement général

Lors du premier démarrage :

1. **Le système détecte un support cloud-init** (image ISO, metadatas d'un cloud, etc.).
2. Cloud-init **lit les fichiers de configuration** fournis (souvent `user-data` et `network-config`).
3. Il exécute les directives pour :

• configurer le clavier, la langue et le fuseau horaire ;

• créer ou modifier des utilisateurs ;

• activer SSH et déployer des clés ;

• installer des paquets ;

• exécuter des commandes personnalisées ;

• écrire des fichiers ou des scripts sur le disque.

Une fois les actions terminées, la machine est prête à l'emploi, déjà configurée selon les besoins.

Créer le fichier `user-data.yaml`

Ce fichier définit les **paramètres de base** appliqués au premier démarrage.

Exemple de laboratoire avec un compte par défaut fourni par l'image :

```
#cloud-config

# Langue et fuseau horaire
keyboard:
  layout: fr
locale: fr_FR.UTF-8
timezone: Europe/Paris

# Gestion des utilisateurs
manage_etc_hosts: true
password: dawan
chpasswd: { expire: False }
ssh_pwauth: True

# Paquets à installer
packages:
  - micro
  - htop
```

Ce fichier peut aussi contenir des directives plus avancées :

- `users` pour créer des comptes spécifiques,
- `runcmd` pour exécuter des commandes au premier démarrage,
- `write_files` pour écrire des fichiers de configuration.

Avant d'utiliser le fichier, on peut vérifier sa validité :

```
sudo cloud-init schema -c user-data.yaml
```

Créer le fichier `network-config.yaml`

Ce fichier utilise ici la configuration réseau Cloud-init **version 2**. Le moteur qui l'applique dépend de l'image ; il ne faut pas supposer que toute installation Debian utilise Netplan. Dans un fichier NoCloud `network-config` séparé, les champs commencent directement à `version`.

Exemple : activer le DHCP sur les interfaces dont le nom commence par `en` :

```
version: 2
ethernets:
  all_interfaces:
    match:
      name: "en*"
```

```
    dhcp4: true
```

Exemple : IP fixe sur une interface secondaire :

```
version: 2
ethernets:
  enp0s3:
    dhcp4: true
  enp0s8:
    dhcp4: false
    addresses:
      - 192.168.56.200/24
```

Générer l'image ISO de configuration

Créer aussi `meta-data.yaml`, qui identifie l'instance :

```
instance-id: debcloud-001
local-hostname: debcloud
```

Une fois les fichiers prêts, on utilise l'outil `cloud-localds` (fourni par le paquet `cloud-image-utils` sur Debian) pour créer une petite image ISO que la VM lira au démarrage :

```
sudo apt install cloud-image-utils
cloud-localds cloud-init.iso user-data.yaml meta-data.yaml
```

Si un fichier réseau est utilisé :

```
cloud-localds --network-config=network-config.yaml cloud-init.iso user-data.yaml
    meta-data.yaml
```

Cette ISO sera montée comme **lecteur CD** dans la machine virtuelle au moment du premier boot.

Utilisation dans VirtualBox ou un hyperviseur

1. Télécharger une image cloud officielle (ex. Ubuntu ou Debian Cloud).
2. Créer une nouvelle VM et **utiliser ce disque comme disque système**.
3. Ajouter un **lecteur CD-ROM** dans lequel on montera `cloud-init.iso`.
4. Démarrer la VM : cloud-init applique automatiquement la configuration.

Le nom du compte par défaut dépend de l'image (souvent `ubuntu` ou `debian`). De nombreuses images n'ont pas de mot de passe de connexion utilisable tant qu'il n'a pas été configuré. Les exemples `dawan` et `admin` sont des valeurs de démonstration pour un laboratoire isolé, à remplacer ; pour un déploiement, préférer une clé publique SSH et un compte verrouillé pour

le mot de passe.

Réinitialiser Cloud-init

Si la configuration doit être rejouée sans recréer la VM, on peut nettoyer les traces précédentes :

```
sudo cloud-init clean --logs
sudo reboot
```

Sur une VM de laboratoire préparée pour cela, ce nettoyage permet de rejouer des étapes d'initialisation. Leur résultat dépend de la source de données et de la configuration ; un changement d'instance-id identifie une nouvelle instance. Préférer un clone neuf pour répéter proprement la démonstration.

Fonctions avancées

Cloud-init offre de nombreuses possibilités :

Exécution de commandes (`runcmd`)

```
runcmd:
  - apt update
  - apt install -y apache2
  - systemctl enable --now apache2
```

Création d'utilisateurs

```
users:
  - name: admin
    plain_text_passwd: admin
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
```

Écriture de fichiers (`write_files`)

```
write_files:
  - path: /etc/motd
    permissions: '0644'
    content: |
      Bienvenue sur cette VM configurée automatiquement avec Cloud-init !
```

Cloud-init sous Debian et Ubuntu

Les images **Ubuntu Cloud** sont prêtes à l'emploi : il suffit d'ajouter l'ISO cloud-init. Sous **Debian**, quelques ajustements peuvent être nécessaires :

- installation du paquet `console-setup` pour le clavier,
- correction du fichier `/etc/locale.gen` pour la francisation,
- choix des paquets et composants nécessaires selon l'image et l'hyperviseur, en suivant la documentation de la distribution.

À retenir

- Cloud-init automatise la **configuration initiale** d'une VM ou d'un serveur cloud.
- Il lit un fichier YAML (`user-data`, `network-config`) et applique les directives au premier démarrage.
- L'image ISO générée par `cloud-localds` remplace les installations manuelles répétitives.
- Compatible avec la plupart des hyperviseurs et clouds modernes.
- Idéal pour **déployer rapidement** plusieurs machines préconfigurées (mot de passe, SSH, paquets, réseau…).

Les fichiers YAML et l'ISO du cours d'origine sont conservés dans `ressources/fichiers`, sans modification : ce sont des exemples historiques de laboratoire, à distinguer des commandes corrigées de cette note.

Documentation : NoCloud — Cloud-init et cloud-localds — Debian.

C08. Personnaliser son terminal

Objectif Adapter l'affichage du shell après avoir acquis les commandes de base.

Complément À découvrir après les bases, selon le rythme du groupe.

Personnaliser le terminal (fichier `.bashrc`)

Le fichier `~/.bashrc` définit l'environnement du shell. C'est ici qu'on peut activer les couleurs et personnaliser le prompt (l'invite de commande).

Activer les couleurs de base :

Ouvre le fichier :

```
nano ~/.bashrc
```

Assure-toi que ces lignes sont présentes (ou décommentées) :

```
alias ls='ls --color=auto'
alias grep='grep --color=auto'
```

Recharge le fichier :

```
source ~/.bashrc
```

Coloriser le prompt (variable PS1)

Le **prompt** est le texte affiché avant le curseur (par exemple `user@debian:~$`). Il est défini par la variable **PS1**.

Exemple simple et coloré :

```
export
    PS1="\[\033[38;5;82m\]\u@\[\033[38;5;214m\]\h\[\033[0m\]:\[\033[38;5;14m\]\w\[\033[0m\]
    \$ "
```

Pour rendre cette personnalisation permanente :

```
echo 'export PS1="\[\033[38;5;82m\]\u@\h:\w\$ "' >> ~/.bashrc
```

Recharge :

```
source ~/.bashrc
```

Générateur de ps1 en ligne : Bash Prompt Generator

Coloriser les pages man (Debian 13)

L'affichage des pages `man` dépend du formateur, du pager et de la configuration du terminal. Les réglages ci-dessous sont des pistes de personnalisation à essayer, pas une étape nécessaire du cours. Il faut réactiver cette fonction en configurant `groff` et `less`.

Ajoute à la fin du `~/.bashrc` :

```
# Couleurs dans les pages man
export PAGER=less
export LESS='-R'
export MANROFFOPT=-c

export LESS_TERMCAP_mb=$'\e[1;31m'
export LESS_TERMCAP_md=$'\e[1;36m'
export LESS_TERMCAP_me=$'\e[0m'
export LESS_TERMCAP_so=$'\e[1;44;33m'
export LESS_TERMCAP_se=$'\e[0m'
export LESS_TERMCAP_us=$'\e[1;32m'
export LESS_TERMCAP_ue=$'\e[0m'
```

Recharge ensuite :

```
source ~/.bashrc
```

> Sur Debian 13, `groff` filtre désormais les séquences de couleurs par défaut pour compatibilité. L'option `MANROFFOPT=-c` désactive ce filtrage et permet à `less` d'afficher les couleurs correctement.

Conserver une personnalisation

`~/.bashrc` est généralement lu pour les shells Bash interactifs non lancés comme shells de connexion ; les fichiers de profil peuvent le charger lors d'une connexion. Avant de le modifier :

```
cp ~/.bashrc ~/.bashrc.sauvegarde
nano ~/.bashrc
bash -n ~/.bashrc
```

`bash -n` contrôle la syntaxe ; il ne garantit pas que les commandes auront l'effet voulu. Ouvrir un second terminal pour vérifier le résultat en conservant la session actuelle.

À retenir

Une personnalisation facilite la lecture, mais les commandes doivent rester compréhensibles sans couleurs ni prompt particulier.

C09. Repères pour poursuivre la préparation LPI 101

Objectif Identifier les notions à revoir pour une préparation plus complète à l'examen.

Repères LPI 101 : 101 à 104.

Complément À découvrir après les bases, selon le rythme du groupe.

Utiliser cette fiche

Le parcours principal privilégie la compréhension et les gestes usuels. Cette fiche rassemble des repères supplémentaires ; elle ne remplace ni l'entraînement ni les leçons détaillées du référentiel LPI.

Reconnaître cpio et dd

`cpio` construit ou extrait une archive à partir d'une liste de noms. Exemple dans un dossier de démonstration contenant `notes.txt` :

```
printf 'Exemple\n' > notes.txt
printf '%s\0' notes.txt | cpio --null -ov > notes.cpio
cpio -it < notes.cpio
```

`dd` copie des données avec des tailles de blocs choisies. Exemple limité à un fichier ordinaire, dans le dossier de travail :

```
dd if=/dev/zero of=exemple.bin bs=1M count=1 status=progress
ls -lh exemple.bin
```

`if` est l'entrée, `of` la sortie et `count` le nombre de blocs. Ne pas transposer cette commande sur un disque : une sortie vers un périphérique peut écraser son contenu.

Reconnaître d’autres filtres

Outil	Rôle	Exemple
`nl`	Numéroter des lignes	`nl /etc/hosts`
`od`	Afficher des données sous une forme numérique	`od -c /etc/hostname`
`paste`	Juxtaposer des lignes de fichiers	`paste noms.txt ages.txt`
`split`	Découper un fichier en morceaux	`split -l 100 grand.txt morceau-`
`sha256sum`	Calculer une empreinte pour contrôler une copie	`sha256sum exemple.bin`

Les noms de fichiers illustratifs doivent être remplacés par des fichiers existants. `sha256sum -c liste.sha256` vérifie des empreintes enregistrées ; la confiance dans l’empreinte publiée dépend aussi de sa provenance.

Compléter la gestion des paquets

```
apt-cache policy bash
apt-cache depends bash
dpkg --verify bash
rpm -V bash
rpm -K paquet.rpm
dnf provides '*/nom-de-fichier'
```

Ces commandes relèvent de distributions différentes : lancer les commandes APT sur Debian et les commandes RPM/DNF dans un environnement RPM. `rpm -V` compare les fichiers installés aux métadonnées enregistrées ; `rpm -K` vérifie les empreintes et signatures d’un paquet selon les clés disponibles.

Étendre ses recherches

`find -perm` sélectionne selon les droits ; `find -user` selon le propriétaire. La base de `locate` est mise à jour avec `updatedb`, selon la configuration de l’implémentation installée.

Préparer sa révision

Reprendre les objectifs officiels 101.1 à 104.7, noter les commandes non reconnues et les essayer dans une VM de laboratoire. Les variantes historiques de GRUB et d’init, les outils propres à XFS et Btrfs et les unités de montage systemd nécessitent des lectures complémentaires pour une préparation exhaustive.

Références : objectifs LPI 101-500 et leçons LPI en français.